

Machine Learning System Design Study Handbook

Hsiang-Yu (Andy) Hsieh

Chapter 0: How I Approach an ML System Design Case

Based on my study of *Machine Learning System Design Interview* (Aminian & Xu, 2023). The framework, explanations and diagrams are my own restatement. Notes marked **Beyond the book** are material I added.

Every case in this handbook, whether feed ranking, ad clicks, friend suggestions or content moderation, follows the same arc: pin down what the product needs, choose how to measure it, build the simplest system that could work, and add complexity only where a measurement shows the simple system falls short. This chapter is that arc as a reusable procedure.

1. The core principle: minimum sufficient design

Choose the smallest architecture that satisfies the current requirements. Add a component only when it fixes a demonstrated quality, latency, scale or operational problem.

Minimum sufficient design



A component belongs in the design only when it traces back to a requirement or a measured limitation.

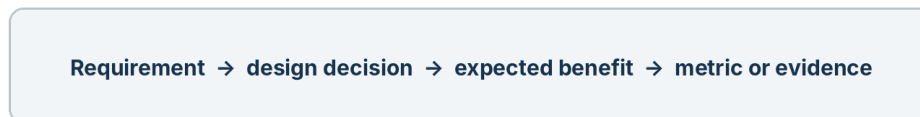


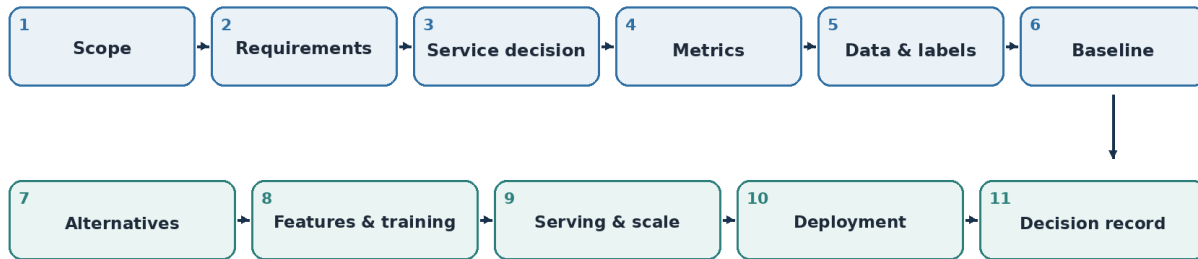
Figure 1: Minimum sufficient design

This isn't minimalism for its own sake. Every component you add is something to train, serve, monitor, debug and explain. A two-stage retrieval + ranking system is the right answer at 10^9 items. At 10^4 items it's overhead, because you can score everything. The interviewer (or the design review) wants to see that you know *which* problem each component solves.

2. The design order

Recurring order of an ML system design case

A checklist for completeness, not a rigid script.



Move a section earlier only when a case-specific constraint changes the architecture.

Figure 2: Design order

#	Step	The question I answer
1	Scope	What exactly is the input, the model's output and the product's action? What's out of scope?
2	Requirements	Scale (users, items, QPS), latency budget, freshness, languages and modalities, cost, fairness and safety constraints
3	Service decision	Which decision does the score feed: rank, filter, threshold, route to a human?
4	Metrics	One primary product metric, guardrails, and offline metrics that predict them
5	Data and labels	What is one training example? Where do labels come from, how noisy are they, when do they arrive?
6	Baseline	The simplest credible model: heuristics, logistic regression, popularity, collaborative filtering
7	Richer alternatives	What specific limitation of the baseline does each fix?

#	Step	The question I answer
8	Features and training	Feature families, freshness, point-in-time correctness, loss, sampling, imbalance
9	Serving and scale	Online vs precomputed, candidate generation, caching, ANN, latency budget per stage
10	Deployment and monitoring	Offline gate → shadow → canary → A/B → ramp; drift, retraining triggers, rollback
11	Decision record	What I chose, and what measured result would make me switch

Back-of-envelope scale

Numbers turn “it’s big” into architecture. For example, with 500M daily active users making about 10 feed requests a day:

$500\text{e}6 \times 10 / 86,400 \text{ s} \approx 58,000 \text{ requests/s}$ on average → plan for ~2–3× at peak

If each request ranks 1,000 candidates, that’s about 6×10^7 model scores per second. That’s why the final ranker only ever sees a small candidate set, and why retrieval uses precomputed embeddings and ANN search.

3. Three lenses on alternatives

For each design area I consider three perspectives:

- **Easy to implement:** a credible baseline for fast validation, debugging and cheap serving. It’s also the yardstick every other option has to beat.
- **More granular or accurate:** a richer option aimed at a specific gap: temporal dynamics, graph structure, multiple modalities, feature interactions, per-behavior outputs.
- **Production-balanced:** the option that best trades off quality, latency, scale, reliability, cost and maintainability. This is often a *combination*: a cheap stage to narrow the candidates, then a rich model on the few that remain.

These are ways of looking at the problem, not a rule to always build three models.

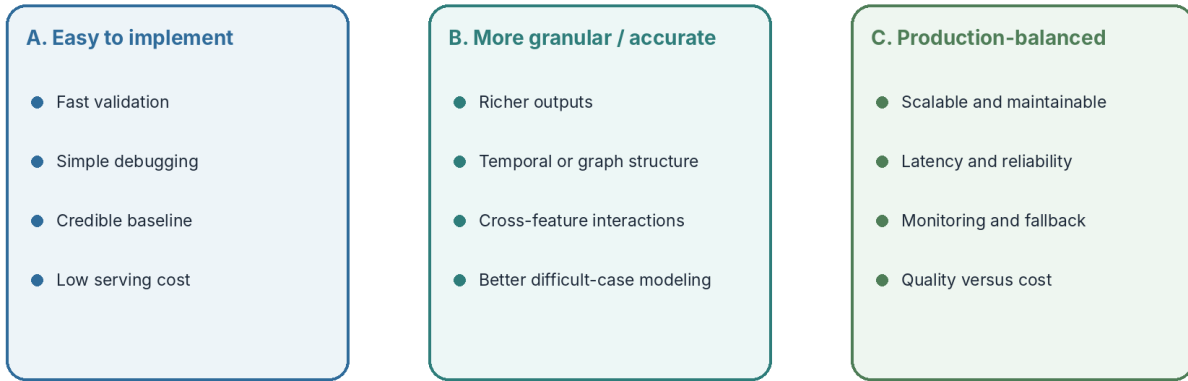
4. How I compare alternatives

For every option I write down six things:

1. **the problem it solves**, e.g. “the LR baseline can’t learn user × ad-category interactions”;
2. **the expected benefit**, e.g. “+0.5–1% relative normalized entropy”;
3. **the cost or risk**: latency, training cost, operational burden, interpretability;
4. **where it fits best**: data size, traffic, freshness needs;
5. **the evidence needed**: an offline metric on a time-based holdout, then an online test;
6. **the switch condition**, stated so it can be measured, e.g. “adopt DCN if it improves offline NE by $\geq 0.5\%$ relative, holds p99 latency under 20 ms, and wins a two-week A/B test on revenue without raising hide rate.”

Point 6 is what separates a design decision from a preference.

Three recurring perspectives for alternatives



These are decision lenses, not a requirement to invent exactly three separate models.

Figure 3: Alternatives lens

How to compare an alternative

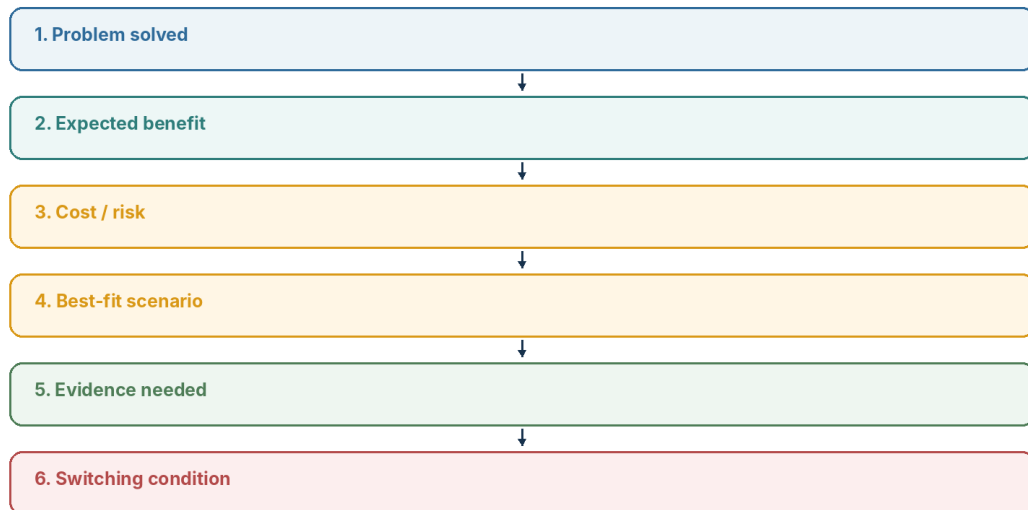


Figure 4: Compare alternatives

5. Model output $\neq$ service action

Separate the model output from the service action



Examples

- category scores $\rightarrow$ harmful score $\rightarrow$ keep / review / remove
- candidate scores $\rightarrow$ ranking policy $\rightarrow$ ordered recommendation list
- event probability $\rightarrow$ threshold policy $\rightarrow$ alert / no alert

Figure 5: Model and service separation

Keep the model's granular predictions separate from the product's decision:

model: $P(\text{like}), P(\text{comment}), P(\text{hide}), \dots$ or $P(\text{violence}), P(\text{nudity}), \dots$
policy: $\text{score} = \sum w_i \cdot p_i$, thresholds, caps, business rules, overrides
action: rank / show / demote / send to review / remove

This separation lets you change product policy (weights, thresholds, frequency caps) without retraining. It keeps the model's outputs meaningful as probabilities. And it gives you per-component diagnostics when something goes wrong. It only works if those probabilities are **calibrated**: a weighted sum of miscalibrated probabilities is not a meaningful utility.

6. Staged production design

Simple and rich models usually coexist as stages:

millions of items $\rightarrow$ cheap retrieval (ANN, graph, rules) $\rightarrow$ ~1,000 candidates
 $\rightarrow$ rich ranker $\rightarrow$ ~50 items $\rightarrow$ policy / diversity / business rules $\rightarrow$ shown

Each stage has its own metric: recall@K for retrieval, nDCG or precision@K for ranking, and the product metric at the end.

7. Serving and deployment

Serving mode, chosen by freshness and latency:

Mode	Use when	Example
online	the input only exists at request time, or freshness matters	feed ranking with the latest interactions

Mode	Use when	Example
precomputed (batch)	inputs change slowly and the set of users/items is predictable	nightly friend suggestions for active users
hybrid	stable parts precomputed, a light fresh re-rank online	cached candidates re-ranked with fresh features

Rollout order. Each stage answers a different question, so I use them in sequence:

1. **Offline evaluation** on a time-based holdout: *is it better on data it hasn't seen?*
2. **Shadow deployment:** the new model scores live traffic without acting. *Does it run correctly (latency, errors, score distributions)?*
3. **Canary:** real actions for a small slice of traffic. *Is it healthy on real users?*
4. **A/B test:** randomized treatment vs control, e.g. 5% / 95%. *Does it causally improve the product metric without hurting guardrails?*
5. **Ramp to 100%,** with rollback available at every step.

Interleaving is a faster, more sensitive alternative to A/B for comparing *rankers*: it mixes two rankings in one list and counts which one's items get clicked. A/B is still needed for product-level effects.

8. Closing statement

"I'd start with the smallest system that satisfies the core requirements, establish a measurable baseline, and add complexity only when a specific product, quality, latency or scale limitation justifies it, stating in advance the measured result that would make me switch."

9. Questions and answers

Q: How do I budget a 45-minute interview?

About 5 minutes on scope and requirements, 5 on metrics, 5 on data and labels, 15 on the model (baseline → richer option), 10 on serving and scale, and 5 on deployment and monitoring. Say the plan out loud at the start so the interviewer can redirect you.

Q: How do I pick the primary metric when the product has several goals?

Choose the one closest to long-term user value that a two-week experiment can still move (e.g. meaningful interactions or accepted connections, not raw clicks). Make the others guardrails with explicit "must not drop more than X%" limits.

Q: When is a heuristic baseline acceptable in production?

When it meets the requirement and the ML gain wouldn't pay for its cost. Popularity plus recency, or a rules-based filter, is sometimes the right launch system, and it becomes the control arm of every later experiment.

Q: What must every design mention, whatever the case?

Point-in-time features (no leakage), a time-based validation split, a calibration check if scores are combined or thresholded, a rollout plan, and monitoring with a defined retraining or rollback trigger.

A production-balanced design can combine alternatives

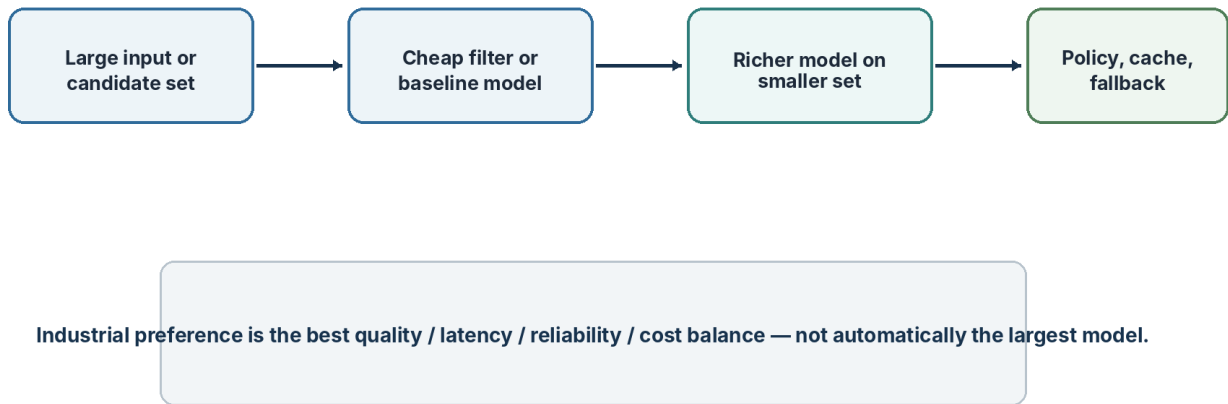
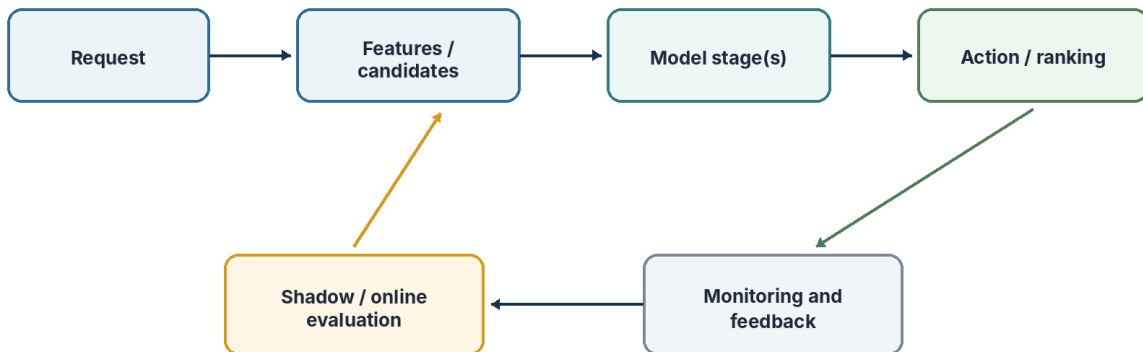


Figure 6: Staged production design

Serving, validation, and monitoring



Use the lowest-risk validation method that can answer the question.

Figure 7: Serving and deployment

Chapter 1: Designing a Large-Scale Social Feed Ranking System

Based on my study of *Machine Learning System Design Interview* (Aminian & Xu, 2023). The explanations, trade-off analysis and diagrams are my own. Notes marked **Beyond the book** are material I added.

The design in one paragraph. Retrieve a few thousand candidate posts from several cheap sources (ANN over two-tower embeddings, the social graph, fresh posts from followed authors). Merge and deduplicate them and attach features. Score them with a **shared-bottom multi-task network** that predicts several behaviors at once: like, comment, share, long dwell, hide, block. Combine those calibrated probabilities into one utility score with product-chosen weights, rank, apply policy rules, and roll out through shadow → canary → A/B.

Ranking-system design map

The chapter moves from product behavior to retrieval, ranking, and safe rollout.



Working assumption

500M DAU and ~200 ms ranking latency are design assumptions, not claims about a specific platform.

Figure 8: Ranking-system design map

1. Scope and requirements

- **Input:** a user opening their feed (user ID plus request context).
- **Model output:** probabilities of several engagement behaviors for each candidate post.
- **Service action:** an ordered list of posts.
- **Goal:** more *meaningful* engagement and a better user experience, not raw clicks.
- **Out of scope:** ads (Chapter 2), content moderation (Chapter 4; assume unsafe posts are already filtered).

Working assumptions (for scale reasoning, not claims about a real platform):

Quantity	Value	Consequence
daily active users	~500M	~58K feed requests/s on average at 10 requests/user/day; plan for 2-3× at peak
latency budget	~200 ms end to end	only a few thousand candidates can be scored by the heavy model
availability	high; degrade gracefully	fall back to a cached or heuristic feed if ranking times out

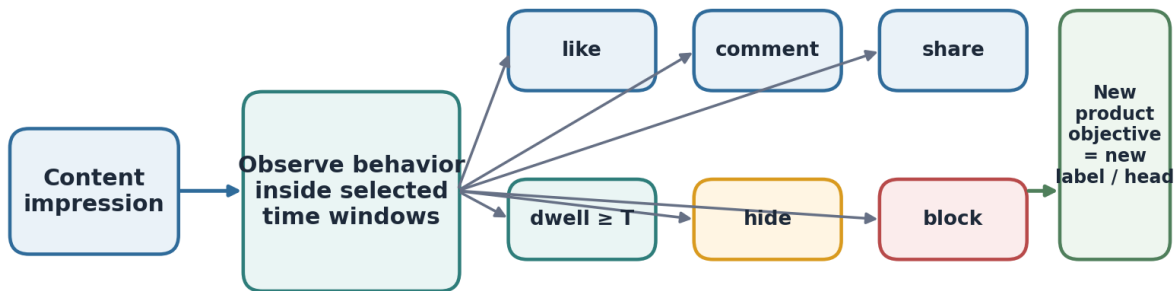
A latency budget I'd propose for the 200 ms:

candidate retrieval (parallel sources)	~30 ms
feature hydration	~40 ms
multi-task ranking (~1-2K candidates)	~80 ms
policy, diversity, dedup	~20 ms
network and slack	~30 ms

2. Labels: one impression, many objectives

From one impression to multiple product objectives

An extensible label system lets one event support several behaviors and future task heads.



Each head gets a binary label: 1 = the behavior happened inside its window, 0 = it did not.

Figure 9: Product objectives and labels

Each impression of a post can produce several binary labels, each defined inside a time window after the impression:

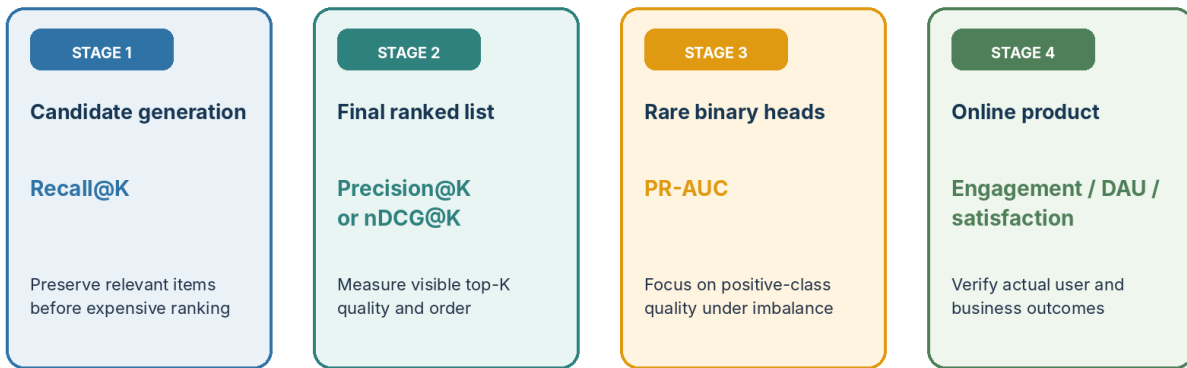
Label	Positive when...	Note
like / comment / share	the action happens within, say, 24 h	comments and shares are rarer and signal stronger interest
dwell $\geq T$	the post stays on screen $\geq T$ seconds	catches passive interest; T is a product choice (e.g. the 70th-percentile dwell)
hide / block	the user hides the post or blocks the author	negative signals, rare but important

Why several labels instead of one “engaged” label? Different behaviors mean different things, and the product wants to weight them differently. A new objective (e.g. “saved”) just becomes a new label and a new head. Labels arrive with **delay**: a share can happen hours later, so training data for day d is only final after the label window closes.

3. Metrics by stage

Choose metrics by stage and product question

No single offline metric represents retrieval quality, final ranking, rare behaviors, and online impact.



Rule of thumb: use a metric only when it matches the stage, ranking depth, label definition, and product decision.

Figure 10: Metrics by stage

Stage	Metric	Question
candidate generation	Recall@K: fraction of the posts the user engaged with that appear in the K candidates	did retrieval keep the good posts?
final ranking	nDCG@K, Precision@K, MRR, mAP	is the visible top of the list well ordered?

Stage	Metric	Question
rare heads (hide, block, share)	PR-AUC, calibration	does each head find its rare positives, and are its probabilities accurate?
overall discrimination	ROC-AUC	does the score separate positives from negatives?
online	meaningful interactions per DAU, time well spent, retention; guardrails: hide rate, reports, latency	does the product improve?

For graded relevance (e.g. share > comment > like > none), nDCG rewards putting the best items first:

$$\text{DCG@}K = \sum_{i=1}^K \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)}, \quad \text{nDCG@}K = \frac{\text{DCG@}K}{\text{IDCG@}K} \in [0, 1].$$

Recall@K can look small and still be fine. If a user engaged with 400 posts last month and $K = 50$, recall is at most 12.5%. That’s a property of the definition, not a bad model. Compare recall between models at the same K , or use a per-session version.

4. Model alternatives

Model	Role	Strength	Weakness
collaborative filtering (matrix factorization)	baseline retrieval	learns community taste from interactions alone	cold start for new users and posts
content-based	cold-start retrieval	works for brand-new posts from text, image and topic	misses “people like you also liked”
two-tower	production retrieval	user and item towers embed independently, so item embeddings are precomputed and searched with ANN	only a dot product links user and item, so no rich cross-features
shared-bottom multi-task DNN	final ranker	full user $\times$ post $\times$ context interactions, one head per behavior	too expensive to run on the whole corpus

Two-tower as a generalization of MF. With only ID embeddings in each tower, a two-tower model is matrix factorization. Adding text, metadata, context and history features makes it a hybrid of collaborative and content-based retrieval. Train with in-batch negatives (softmax over the other items in the batch) and correct for the fact that popular items appear more often as negatives, by subtracting $\log q_j$ (the item’s sampling probability) from its logit (Yi et al., 2019).

Shared bottom vs mixture of experts. In a shared-bottom model all heads read one shared representation. That’s efficient and helps rare tasks borrow signal from common ones, but tasks that conflict (like vs hide) can pull the shared layers in opposite directions (negative transfer). **MMoE** (Ma et al., 2018) replaces the shared bottom with several expert networks and a per-task softmax gate that mixes them, so each task gets its own blend. I’d start with shared-bottom and switch to MMoE if per-task validation shows one head degrading when others are added.

Model alternatives: baseline, richer retrieval, and final ranking

The alternatives solve different stages and can be combined rather than treated as one winner.

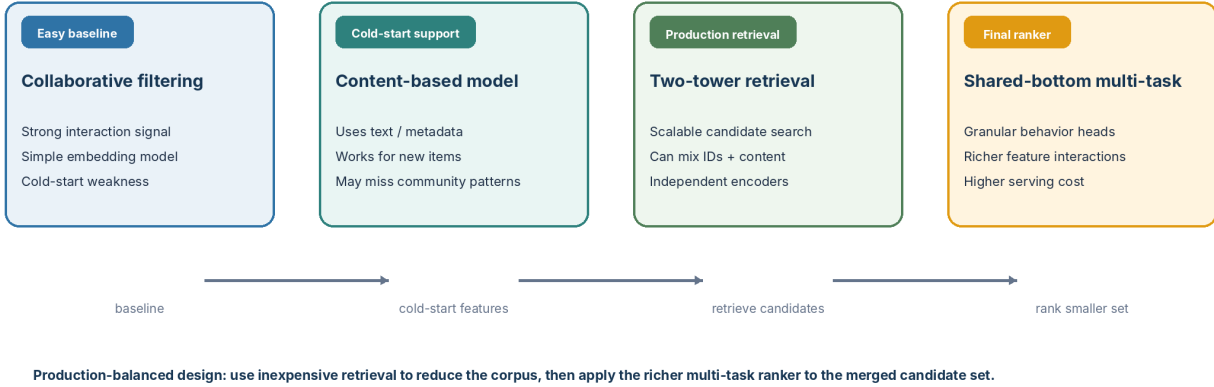


Figure 11: Model alternatives

5. Architecture and the serving score

Inputs: user features, post features, history and context, and **user-author relationship features** (friend or follower, closeness, interaction frequency, mutual friends). Relationship signals are *inputs*, not a separate output.

Two kinds of weights, kept separate:

- **training weights** $\lambda_i \geq 0$ balance the task losses: $L = \sum_i \lambda_i L_i$ (binary cross-entropy per head). They're an optimization tool;
- **serving weights** w_i express product value and can be **negative**:

$$\text{score}(u, p) = \sum_i w_i \hat{P}_i(u, p), \quad \text{e.g. } w = \{\text{like} : 1, \text{comment} : 4, \text{share} : 6, \text{hide} : -20, \text{block} : -50\}.$$

Example. Post A has $P(\text{like}, \text{comment}, \text{share}, \text{hide}) = (0.10, 0.02, 0.01, 0.005)$, so its score is $0.10 + 0.08 + 0.06 - 0.10 = 0.14$. Post B is $(0.20, 0.01, 0.005, 0.03)$: $0.20 + 0.04 + 0.03 - 0.60 = -0.33$. B gets twice the likes but ranks far below A, because of its hide risk.

Warning. > **The sum only means something if each $\hat{P}_i$ is calibrated.**

Heads trained on downsampled data, or with different loss weights, can be systematically over- or under-confident. Check reliability curves per head, and recalibrate (Platt or isotonic scaling on a recent holdout) before combining. Otherwise changing λ_i silently changes the ranking.

6. Data, features and freshness

Family	Examples	Freshness
user	ID embedding, language, location, long-term topic interests	daily batch

Family	Examples	Freshness
post	text/image embeddings, topic, author, age of post	computed once at upload
interaction history	last N posts engaged, per-topic counts in 1 h / 1 d / 7 d windows	near-real-time (streaming)
relationship	friend/follow, closeness, likes on this author in 30 d	hourly-daily
engagement so far	post's likes/comments in the last hour (velocity)	real-time
context	time of day, device, network, session depth	request time

An **aggregated** feature summarizes many events (7-day like count). A **delayed** feature is computed asynchronously and may lag. One feature can be both. Every feature must be logged **as it was at serving time** (point-in-time correctness). Training on today's value of a feature for yesterday's impression is leakage.

7. Training

- **Examples:** one row per (user, post, impression time) with all heads' labels. Non-impressed posts aren't negatives.
- **Loss:** $L = \sum_i \lambda_i L_i$. Choose λ_i by per-head validation metrics and the final ranking metric. Principled options include uncertainty weighting (Kendall et al., 2018) and GradNorm (Chen et al., 2018), which equalizes the gradient magnitudes of different tasks.
- **Imbalance:** positives for share and block are rare. Downsample negatives for efficiency, then correct the probabilities (see Chapter 2, §3).
- **Position bias:** users click what's on top, whatever its quality. Add position as a training feature through a small separate tower, and set it to a fixed value at serving time.

8. Candidate generation, merging and ranking

Several sources run **in parallel**, each with a budget:

Source	Finds	Budget (example)
ANN over two-tower embeddings	posts similar to the user's interests	500
social graph	recent posts from friends and followed accounts	800
trending / fresh	new posts with high early engagement	200
other specialized sources	groups, topics followed	200

Merge, **deduplicate by post ID**, and keep a feature for *which sources* found each post. A post found by both ANN and the graph is a stronger candidate, so that agreement shouldn't be thrown away. Then hydrate features and rank. Tune the source budgets offline by recall@K per source, and online by the share of final impressions each source contributes.

9. The indexing service

The indexing service builds and refreshes the searchable structures that retrieval queries. **Vector quantization** / **product quantization** are compression techniques *inside* it, not separate

Shared-bottom multi-task ranking architecture

Training-loss weights and serving-time utility weights solve different problems.

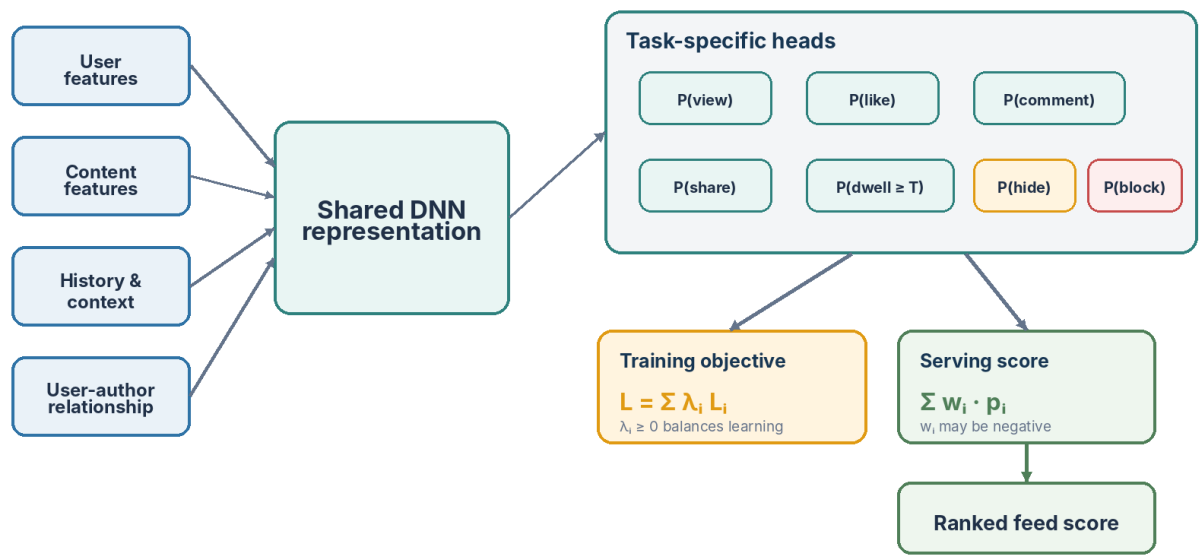
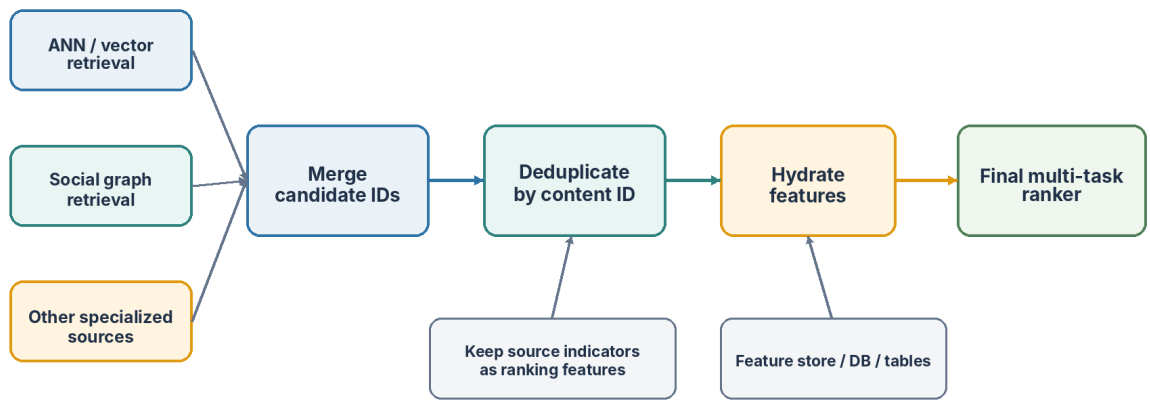


Figure 12: Multi-task architecture

Candidate generation, merge, hydration, and final ranking

Multiple retrieval sources return IDs; the ranking service scores one deduplicated, enriched set.

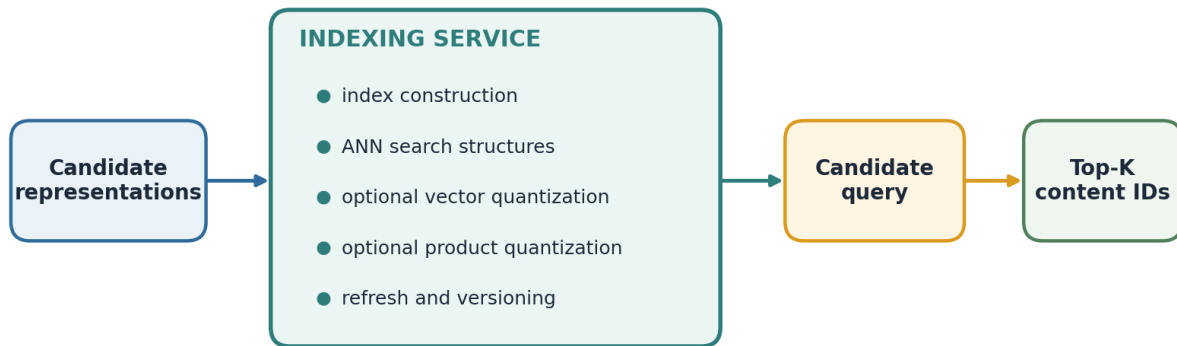


A duplicate retrieval hit is not wasted information: agreement across sources can become a useful ranking feature.

Figure 13: Candidate generation and serving

Indexing service as a retrieval abstraction

Vector quantization and product quantization are optional techniques inside the retrieval service boundary.



Service boundaries

Feature store = reusable features | Indexing service = searchable representations | Ranker = enriched candidate scoring

Figure 14: Indexing service

objectives.

Why compression matters: $1\text{B posts} \times 128\text{-dim float32 embeddings} = 10^9 \times 128 \times 4\text{ B} \approx 512\text{ GB}$. Product quantization with 16 one-byte codes per vector brings that to $\approx 16\text{ GB}$, small enough to fit in memory on a few machines, at a small cost in recall. HNSW graphs give high recall at low latency. IVF-PQ trades some recall for much less memory.

Service boundaries:

- **feature store:** reusable features for training and serving, with point-in-time snapshots;
- **indexing service:** embeddings $\rightarrow$ ANN index, refresh and versioning;
- **candidate-generation service:** queries the index and other sources;
- **ranking service:** hydrates features and scores the merged candidates.

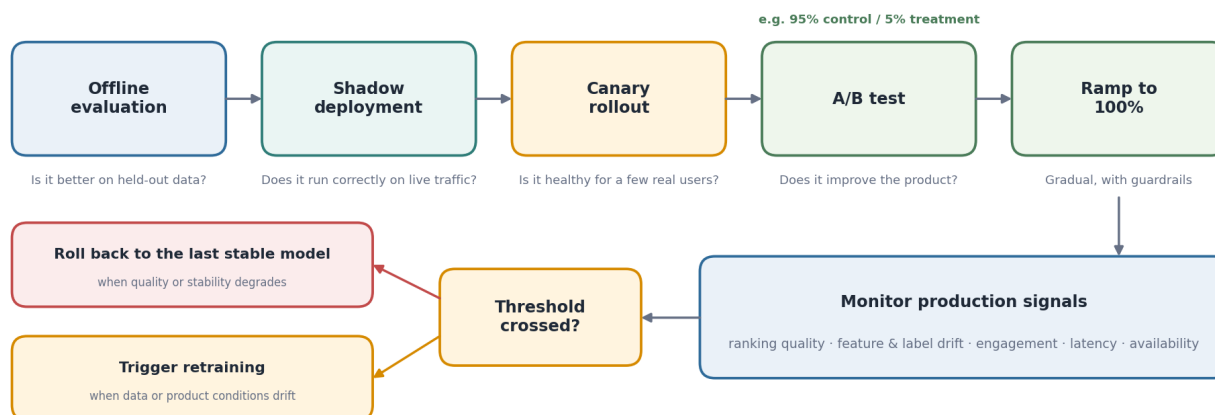
10. Validation and rollout

1. **Offline gate:** better nDCG and per-head PR-AUC on a time-based holdout, with calibration within tolerance.
2. **Shadow:** score live traffic without serving it; check latency, errors and score distributions.
3. **Canary:** serve a small slice of traffic; watch crash, timeout and hide rates.
4. **A/B test:** e.g. 5% treatment vs 95% control for about 2 weeks (to cover weekly cycles and novelty effects); primary metric plus guardrails.
5. **Ramp** 5% $\rightarrow$ 25% $\rightarrow$ 50% $\rightarrow$ 100%, with rollback available at every step.

An A/B test asks *does it make the product better?* A canary asks *is it safe to run?* They answer different questions, which is why both are in the sequence.

Validation, safe rollout, and monitoring

Each stage answers a different question; rollback stays available at every step.



Retraining options: fine-tune with regularization · add adapters / new layers · replay a sample of older data to limit forgetting

Figure 15: Validation and monitoring

11. Monitoring and continual learning

Monitor ranking quality (online nDCG proxies, CTR by position), per-head calibration, feature drift (population stability index on key features), label drift (base rates), engagement, latency and availability.

Retraining can be scheduled (e.g. daily fine-tuning on the latest window) or **triggered** (calibration error or drift beyond a threshold). Options for updating without forgetting:

- **regularization** toward the previous weights (e.g. an L2 penalty on the change, or EWC);
- **adapters or new layers** trained on new data while the core is frozen;
- **replay** of a sample of older data. This reduces forgetting but shifts the training distribution toward the past, so keep the replay ratio small and time-aware.

12. Reference architecture

Offline / async: interaction logs → label generation → feature pipelines → feature store → training → model registry; post embeddings → ANN index. **Online:** request → parallel candidate sources → merge + dedup + hydrate → multi-task ranker → utility score → policy and diversity → feed. Impressions and interactions are logged back for evaluation and training.

13. Decisions and when I'd change them

Area	Starting choice	Switch when (measured)
objective	multi-behavior heads + utility score	a single "meaningful interaction" head matches online results in an A/B test
retrieval	two-tower ANN + graph + fresh sources	one source alone reaches $\geq 95\%$ of the multi-source recall@K at lower latency
ranker	shared-bottom multi-task	a head loses $> 1\%$ relative PR-AUC when trained jointly (negative transfer) → MMoE

Area	Starting choice	Switch when (measured)
serving weights w_i	set by product, tuned by A/B	an experiment on weight vectors finds a better trade-off on the primary metric vs hide rate
index	HNSW or IVF-PQ	exact search fits the latency budget (small corpus)
rollout	offline $\rightarrow$ shadow $\rightarrow$ canary $\rightarrow$ A/B $\rightarrow$ ramp	a low-risk change (e.g. a weight tweak) can skip shadow
retraining	daily fine-tune + drift triggers	calibration error stays within tolerance for weeks $\rightarrow$ weekly

14. Questions and answers

Q: How should the multi-task loss weights λ_i be chosen?

Start equal. Adjust so no head's validation metric is worse than when trained alone, and use uncertainty weighting or GradNorm if the heads' loss scales differ a lot. Judge by the final utility ranking (nDCG of the combined score), not by any single head.

Q: When would MMoE beat shared-bottom?

When tasks are weakly related or conflicting (engagement vs hide/block) and there's enough data per task. Symptom: adding a task hurts another task's metric. With few, closely related tasks, shared-bottom is simpler and just as good.

Q: Which features need real-time freshness?

Session context, the user's last few interactions, and post engagement velocity. These change within minutes and carry most of the short-term signal. Long-term interests and embeddings can be batch-updated daily.

Q: How should candidate-source budgets be set?

Offline, by each source's marginal recall: how many engaged posts it finds that no other source finds. Online, by the fraction of final impressions and engagements each source contributes. Shrink sources whose candidates rarely survive ranking.

Q: Which offline results should gate an online test?

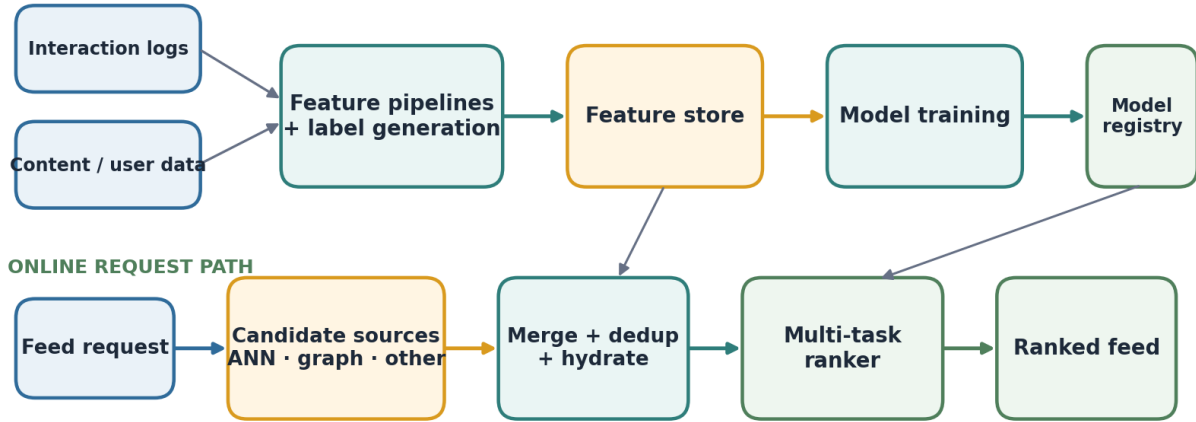
No regression on any head's PR-AUC, an improvement in the combined score's nDCG@K on a time-based holdout, calibration error below a set tolerance, and inference latency within budget.

Q: What should trigger retraining?

Calibration error (e.g. expected calibration error) or base-rate drift beyond a threshold, a drop in online engagement relative to control, or a population stability index above about 0.2 on important features.

One defensible large-scale ranking architecture

A hybrid retrieval-and-ranking system under the current assumptions.
OFFLINE / ASYNCHRONOUS



Feedback loop: impressions and interactions return to logs, enabling evaluation, monitoring, and future updates.

Figure 16: Reference architecture

Chapter 2: Predicting Ad Clicks on a Social Media Platform

Based on my study of *Machine Learning System Design Interview* (Aminian & Xu, 2023). The explanations, trade-off analysis and diagrams are my own. Notes marked **Beyond the book** are material I added.

The design in one paragraph. For each eligible ad, predict a **calibrated** click probability $\hat{p} = P(\text{click} \mid \text{user, ad, context})$. The auction ranks ads by expected value, $\text{bid} \times \hat{p}$, so the probability itself, not just its ordering, drives revenue. Start with logistic regression on crossed features. Move to a learned-interaction model (DCN or DeepFM) when it measurably beats LR. Use two-tower retrieval only when the ad inventory is too large to score everything. Keep frequency caps and hide/block rules in the serving policy, not in the model. Validate with point-in-time data, a shadow run, a canary and an A/B test.

1. Scope and product policy

- **Input:** a user, a slot in their feed, and the set of **eligible** ads (targeting, budget and policy checks already passed).
- **Model output:** $\hat{p}$ for each eligible ad.
- **Service action:** choose which ad fills the slot.

Why calibration matters more here than in feed ranking: in a cost-per-click auction the platform's expected revenue from showing an ad is

$$\text{eCPM} = \text{bid}_{\text{CPC}} \times \hat{p} \times 1000.$$

If $\hat{p}$ is $2\times$ too high for one advertiser's ads, they win auctions they shouldn't, and the pricing is wrong. **Ranking quality (AUC) isn't enough: the numbers have to be right.**

Ad-click prediction design map

The chapter moves from product behavior and labels to ranking, calibration, and safe rollout.



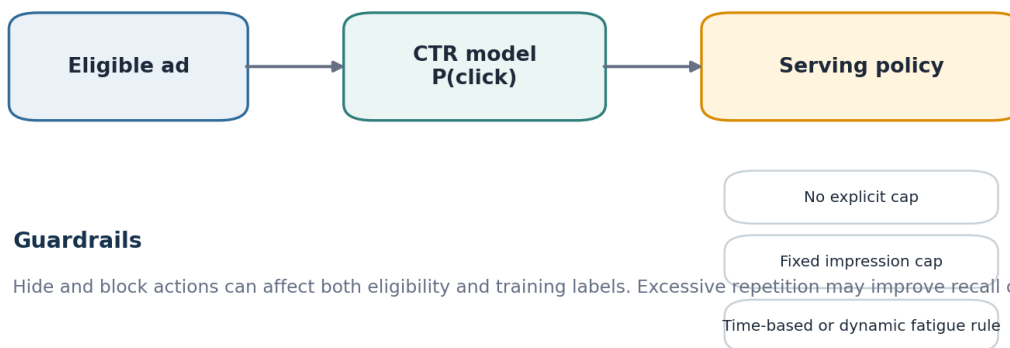
Minimum-scope principle

Start with a calibrated Logistic Regression baseline. Add interaction modeling or retrieval complexity only when data and scale justify it.

Figure 17: Ad-click prediction design map

Repeated exposure is a product decision

The model predicts click probability, but serving policy decides how often the same ad may reappear.



Guardrails

Hide and block actions can affect both eligibility and training labels. Excessive repetition may improve recall or create user fatigue.

Figure 18: Repeated exposure policy

Repeated exposure is a policy decision, not a model decision. The model estimates the click probability. The serving policy decides how often the same ad may reappear: no cap, a fixed impression cap (e.g. 3 per user per day), or a time-decay fatigue rule. Hide and block actions remove the ad (or advertiser) for that user and also become training signals. Include “times this user has seen this ad” as a model feature so the model can learn fatigue, but enforce hard caps in policy.

2. Data and labels

Data and label construction

Clicks are clear positives. Viewed-but-unclicked impressions are ambiguous.

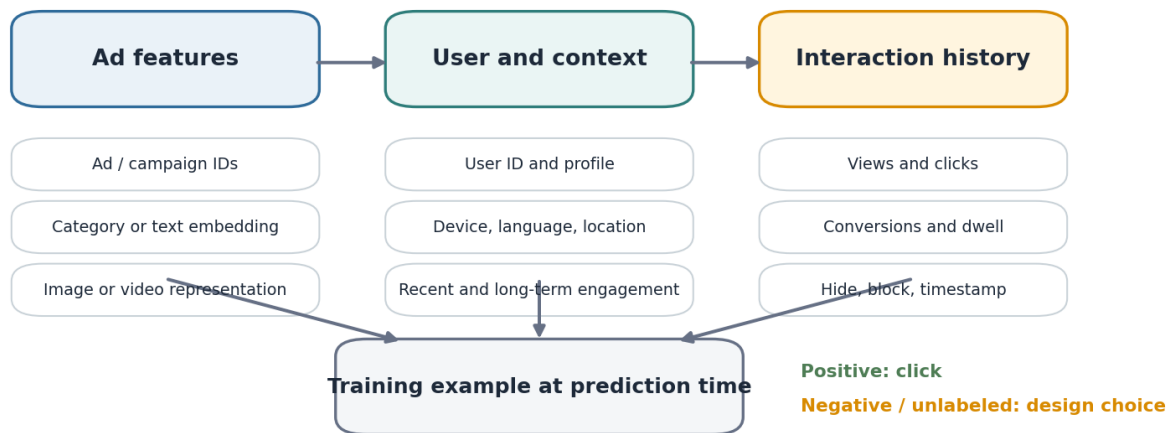


Figure 19: Data and labels

Family	Examples
ad / campaign	ad ID, advertiser ID, category, creative text/image embeddings, campaign age
user / context	user ID, demographics, language, device, location, time of day
interaction history	user's CTR on this category or advertiser, recent clicks, conversions, dwell
impression	position, times already shown, surface

One training example = one impression, with features as they were at impression time.

- **Positive:** a click within a window (e.g. 30 min) after the impression.
- **Negative:** an impression with no click in the window. This is the transparent default.
- **The ambiguity:** “viewed but not clicked” isn’t the same as “not interested”. The user may have scrolled past without seeing the ad, or clicked a day later. Options:
 - require a minimum on-screen time before counting a negative;
 - down-weight uncertain negatives;
 - treat non-clicks as *unlabeled* and use positive-unlabeled (PU) learning.

Start transparent and add these only if hidden positives measurably hurt calibration or ranking.

- **Delayed feedback:** clicks and especially conversions arrive late. Training on too-recent data mislabels future positives as negatives. Wait for the window, or model the delay explicitly.

3. Metrics

Metrics answer different questions

No single offline metric represents ranking quality, calibration, revenue, and user experience.

Probability quality	Binary cross-entropy	Are predicted probabilities numerically useful?
Relative probability quality	Normalized cross-entropy	Did the model beat a clearly named baseline?
Discrimination	ROC-AUC / PR-AUC	Can the score separate clicks from non-clicks?
Online product	CTR / conversion / revenue lift	Did the system improve business outcomes?
User guardrail	Hide or block rate	Did monetization gains harm user experience?

Normalized cross-entropy is interpretable only when the comparison baseline is explicit: constant historical CTR, current production model, or Logistic Regression.

Figure 20: Metrics

Question	Metric
are the probabilities accurate?	log loss (binary cross-entropy); calibration ratio $\sum \hat{p} / \sum y$ (should be ≈ 1)
better than a named baseline?	normalized entropy
does the score separate clicks from non-clicks?	ROC-AUC; PR-AUC because clicks are rare
does the business improve?	CTR, conversions, revenue per 1,000 impressions
is the user experience protected?	hide/block rate, ad load, session length (guardrails)

Normalized entropy (He et al., 2014) divides the model's log loss by the log loss of always predicting the average CTR $\bar{p}$:

$$\text{NE} = \frac{-\frac{1}{N} \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]}{-[\bar{p} \log \bar{p} + (1 - \bar{p}) \log(1 - \bar{p})]}.$$

Lower is better, and $\text{NE} < 1$ means the model beats the constant predictor. Example: with $\bar{p} = 2\%$ the denominator is 0.098 nats, so a model with log loss 0.090 has $\text{NE} = 0.918$. Always say which baseline you normalize by: the constant CTR, the production model, or the LR baseline.

Note. > Beyond the book: correcting for negative downsampling

Clicks are ~1-2% of impressions, so it's common to keep only a fraction w of negatives (e.g. $w = 0.1$). The model then learns inflated odds, by a factor $1/w$. Recover the true probability with

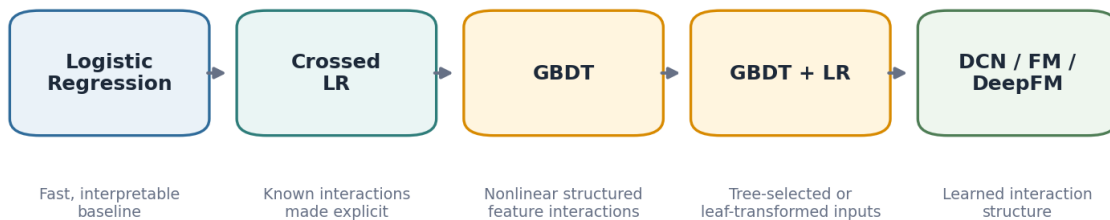
$$q = \frac{\hat{p}}{\hat{p} + (1 - \hat{p})/w}.$$

For example, $\hat{p} = 0.10$ with $w = 0.1$ becomes $q \approx 0.011$. Skipping this step breaks the eCPM auction even when AUC is unchanged.

4. Model alternatives

A practical model progression

The options represent increasing interaction capacity, not a mandatory sequence.



Starting experiment

Use Logistic Regression as the anchor, then compare DCN and DeepFM when automatic feature interaction learning is valuable.

Figure 21: Model progression

Model	How it handles feature interactions	Strength	Cost
logistic regression	none unless you add them by hand	fast, interpretable, calibrated by construction, easy online updates	misses interactions
LR + feature crosses	hand-made crosses (user_country × ad_category), hashed into buckets	captures the crosses you know about	combinatorial feature explosion; only crosses you thought of
GBDT	tree splits learn nonlinear interactions	strong on dense features	awkward with sparse IDs; slow to update incrementally

Model	How it handles feature interactions	Strength	Cost
GBDT + LR	tree leaves become one-hot features for LR (He et al., 2014)	automatic cross discovery + LR's easy updates	two-stage pipeline
factorization machine	every pair i, j gets weight $\langle \mathbf{v}_i, \mathbf{v}_j \rangle$ from low-rank embeddings	pairwise interactions even for rare pairs, $O(kn)$ time	only second-order
DeepFM	FM component + a deep MLP, sharing embeddings	low- and high-order interactions, no manual crosses	more latency and tuning
DCN	cross layers $\mathbf{x}_{l+1} = \mathbf{x}_0 \mathbf{x}_l^\top \mathbf{w}_l + \mathbf{b}_l + \mathbf{x}_l$ build explicit crosses of degree $l + 1$, plus a deep path	explicit, bounded-degree crosses at low cost	more complex than LR
xDeepFM	compressed interaction network: explicit vector-wise crosses	richer explicit interactions	heavier still

Factorization machine:

$$\hat{y} = w_0 + \sum_i w_i x_i + \sum_{i < j} \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j.$$

The pairwise term can be rewritten as $\frac{1}{2} \sum_f [(\sum_i v_{if} x_i)^2 - \sum_i v_{if}^2 x_i^2]$, which costs $O(kn)$ instead of $O(kn^2)$. Because each feature has its own embedding, the model can score a pair (user_country, ad_category) it never saw together in training.

My plan: LR (with a few known crosses) as the anchor and the control arm. DCN and DeepFM as the challengers. Adopt one if it improves NE by a meaningful margin (e.g. $\geq 0.5\%$ relative, which is significant at ad scale), stays calibrated after correction, and fits the latency budget.

5. Where the two-tower model belongs

A two-tower model scores with a dot product between a user embedding and an ad embedding. That makes it great for **retrieval** (ad embeddings precomputed, ANN search over millions of ads) and weak as the **final CTR model**: there are no cross-features, and its scores aren't calibrated probabilities. So:

- **small inventory** (thousands of eligible ads per request): skip retrieval and score them all with the CTR model;
- **large inventory**: two-tower / ANN retrieval $\rightarrow$ a few hundred candidates $\rightarrow$ the rich, calibrated CTR model.

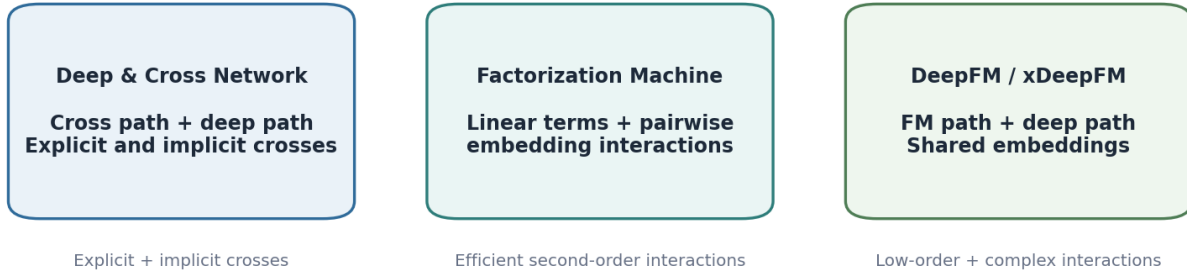
Candidate sources (ANN, collaborative filtering, eligibility/campaign rules, others) are merged and deduplicated, and each ad keeps a feature for which sources found it.

6. Keeping the model fresh

Ads churn fast: new campaigns every day, and user interests and seasonality shift. Stale models lose calibration first. Triggers for an update: calibration ratio drifting from 1, campaign turnover, feature distribution shift, falling online metrics. Update options, from lightest to heaviest:

How richer models learn interactions

Each family encodes user-ad relationships differently.



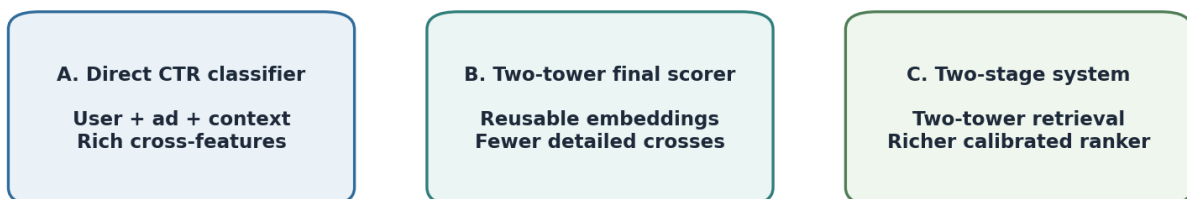
Selection question

Which interaction patterns matter, how much interpretability is needed, and what serving latency is acceptable?

Figure 22: Interaction models

Where should a two-tower model sit?

Retrieval and final CTR ranking may need different representations.



Starting recommendation

Use the two-tower model for scalable candidate retrieval and preserve a richer classifier for calibrated final ranking.

Figure 23: Two-tower placement

Candidate generation and final ranking

Candidate sources may be combined, then deduplicated and enriched before CTR scoring.

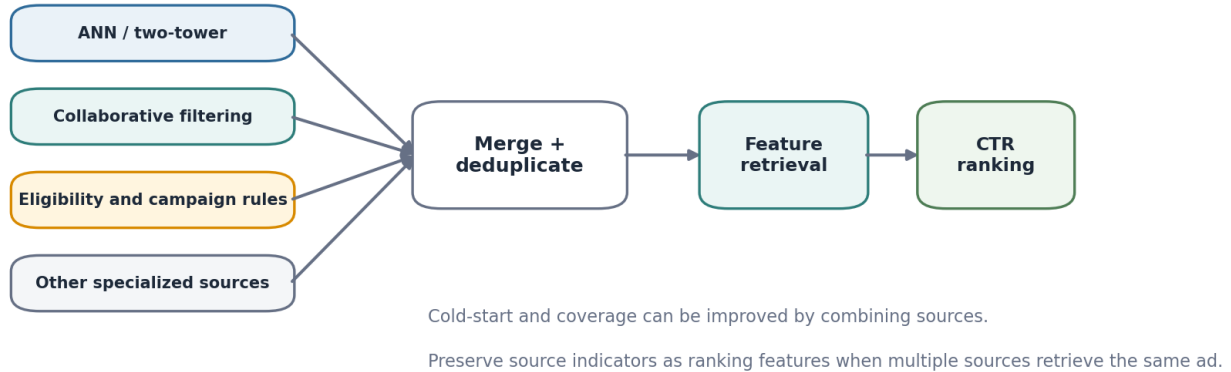
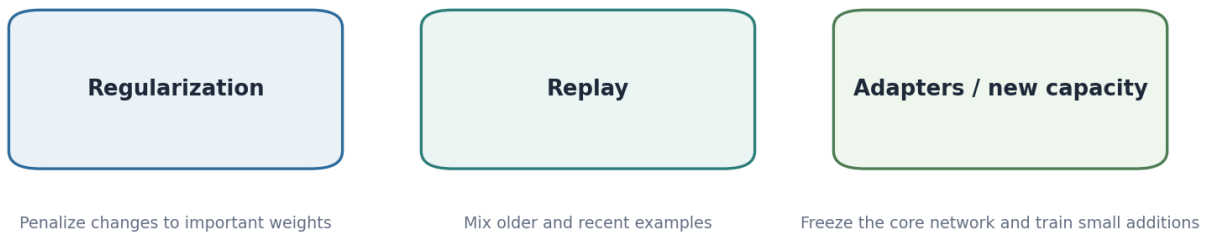


Figure 24: Candidate generation

Continual learning is a drift response

Choose the smallest update strategy that addresses observed drift without unnecessary forgetting.



Trigger examples

CTR calibration drift, campaign turnover, changing user interests, feature distribution shift, or declining online metrics.

They can be used alone or combined; pick the smallest one that fixes the measured problem.

Figure 25: Continual learning

1. **frequent retraining / warm-start fine-tuning** on the latest window (often daily or more);
2. **regularization** toward the previous weights, to limit forgetting;
3. **replay** a sample of older data;
4. **adapters / new capacity** for new ad types.

Use the lightest option that fixes the measured drift.

7. Validation, rollout and calibration

Validation, rollout, and probability calibration

Shadow, canary, and A/B testing provide different evidence.

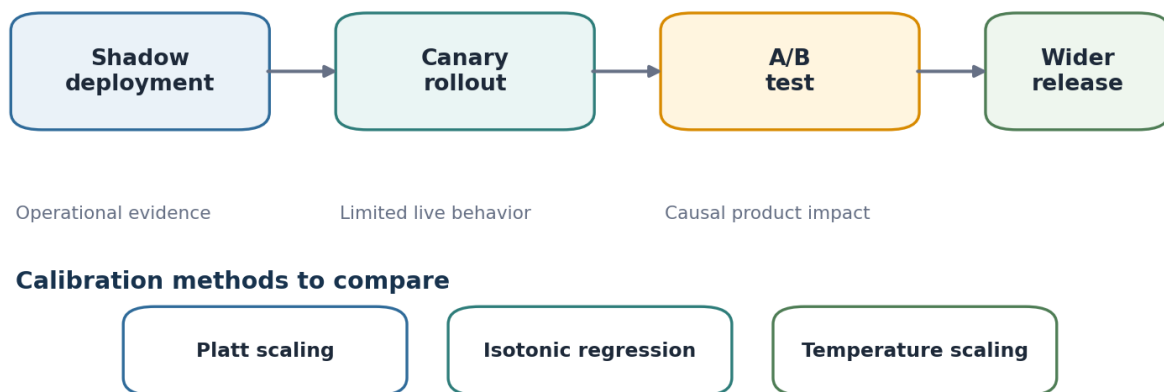


Figure 26: Validation and calibration

- **Shadow:** the new model scores live requests without affecting auctions. Compare score distributions, calibration and latency with production.
- **Canary:** it serves a small slice. Check for errors, spend anomalies and advertiser complaints.
- **A/B test:** measures causal impact on CTR, revenue and hide rate. Watch **budget effects:** treatment and control share advertiser budgets, which can bias results, so budget-split or advertiser-level randomization may be needed.
- **Calibration methods**, all fitted on a recent holdout:

Method	Fits	Use when
Platt scaling	$\sigma(a \cdot \text{logit} + b)$	miscalibration is a smooth shift or scale
isotonic regression	any monotone map	lots of data, irregular miscalibration
temperature scaling	a single T dividing logits	neural models that are over-confident

Prevent future-data leakage

Every feature and label must be computed relative to a prediction-time cutoff.

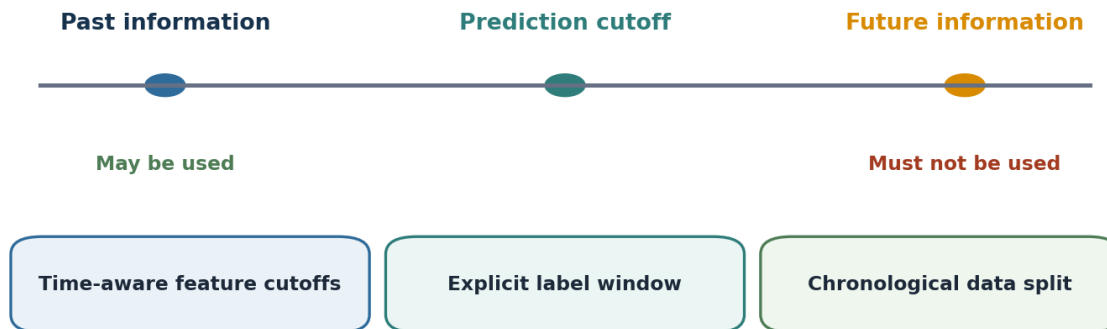


Figure 27: Leakage prevention

8. Leakage

Every feature and label is computed relative to the **impression time**:

- features use only events before the impression (e.g. “user’s clicks in the last 7 days *before* this impression”);
- labels use an explicit window after it;
- train/validation/test are split **chronologically**, e.g. train on days 1-28, validate on day 29, test on day 30.

A random split lets the model see tomorrow’s behavior of the same campaign, and the offline metrics come out better than what you’ll ever see in production.

9. Reference architecture

Online: request → eligibility + frequency policy → candidate retrieval (if the inventory is large) → feature hydration → CTR model + calibration → auction ($\text{bid} \times q$) → ad served. **Offline:** impression/click logs → label construction with windows → point-in-time training data → training + chronological evaluation → model registry → staged rollout.

10. Decisions and when I’d change them

Area	Starting choice	Switch when (measured)
exposure	fixed frequency cap + hide/block rules	a dynamic fatigue model improves revenue at an equal or lower hide rate in an A/B test

Area	Starting choice	Switch when (measured)
negatives	all non-clicked impressions (with a minimum view time)	PU or weighting improves calibration on a trusted holdout
baseline richer model	LR + known crosses DCN or DeepFM	- $\geq \sim 0.5\%$ relative NE gain, calibrated, within p99 latency
retrieval	none (score all eligible ads)	eligible ads per request exceed what the CTR model can score in budget
calibration	Platt on a daily holdout	the reliability curve is non-sigmoid $\rightarrow$ isotonic
freshness	daily warm-start retraining	NE decays measurably within a day $\rightarrow$ more frequent or online updates
leakage	point-in-time features, chronological splits	never optional

11. Questions and answers

Q: Why optimize log loss rather than AUC?

AUC ignores calibration: multiplying every prediction by 2 leaves AUC unchanged and breaks the auction. Log loss (and NE) penalizes both poor ranking and wrong probabilities.

Q: How do I handle a brand-new ad with no history?

Use content features (creative embeddings, category, advertiser history) so the model can generalize, add a small exploration budget (e.g. Thompson sampling on an uncertainty estimate) so new ads get impressions, and back off to advertiser- or category-level CTR priors.

Q: Which normalized-entropy baseline should I report?

State it explicitly. The constant average CTR is standard (NE < 1 means “better than knowing nothing about the user”). For launch decisions, compare with the current production model.

Q: Should frequency be a feature or a rule?

Both. The feature lets the model learn that the 5th showing is less likely to be clicked. The rule enforces a hard limit that the product guarantees users, whatever the model says.

Q: What does the A/B test need beyond CTR?

Revenue and advertiser value (conversions, cost per acquisition), user guardrails (hide/block rate, session time), and a check that treatment and control didn’t compete for the same budgets.

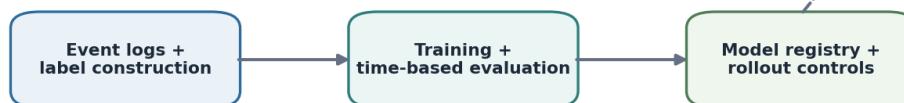
Reference architecture

One reasonable production-balanced design under the current assumptions.

ONLINE SERVING



OFFLINE LEARNING AND VALIDATION



Starting recommendation: two-tower or ANN retrieval for scale, a richer calibrated CTR model for final ranking, and explicit frequency and hide/block guardrails.

Figure 28: Reference architecture

Chapter 3: People You May Know

Based on my study of *Machine Learning System Design Interview* (Aminian & Xu, 2023). The explanations, trade-off analysis and diagrams are my own. Notes marked **Beyond the book** are material I added.

The design in one paragraph. People You May Know (PYMK) is **link prediction**: given the social graph up to now, which non-connected pairs are likely to connect *and* both value the connection? Generate a bounded pool of candidates, mostly friends-of-friends. Score them with a simple supervised model on graph, profile and activity features. Precompute and cache the lists for active users, refreshing lazily. Promote a **temporal GNN** only if it measurably beats the static model with recency features.

1. Scope and scale

- **Input:** a user u opening a PYMK surface.
- **Model output:** P (a connection between u and v forms and is accepted within Δt) for candidate users v .
- **Service action:** a short ranked list (e.g. 10–20 people), excluding existing connections, blocked users and people who opted out.

Working assumptions: ~1B total users, ~300M daily active users, ~1,000 connections per user on average. The arithmetic drives the whole design:

Quantity	Estimate	Consequence
2-hop (friend-of-friend) paths per user	$1,000 \times 1,000 = 10^6$ (fewer unique people due to overlap; far more for high-degree users)	too many to score with a rich model

Quantity	Estimate	Consequence
scoring all 2-hop pairs for every DAU daily	$3 \times 10^8 \times 10^6 = 3 \times 10^{14}$	impossible, so the pool must be capped
cap at 2,000 candidates per user (e.g. by mutual-friend count)	6×10^{11} scores/day $\approx 7\text{M/s}$	feasible only for a cheap scorer
rich model on the top 200 3-hop	6×10^{10} /day $\approx 0.7\text{M/s}$ $\sim 10^9$	feasible, in batch only via sampling (random walks), never exhaustive

So: **exclude 1-hop, cap 2-hop, deduplicate, narrow cheaply, and score richly only at the end.**

2. Metrics

Metric	Measures	Pitfall
requests sent / received	activity	rewards spammy, low-quality outreach
accepted requests	mutual relevance	may still miss long-term value
completed connections that go on to interact	real network growth	sparse and delayed
Precision@K, mAP (offline)	quality of the visible list	offline proxy only
PR-AUC (offline)	separating rare positive edges	ignores list position

Starting choice: gate models offline with Precision@K / mAP and PR-AUC on future edges. Online, optimize **accepted connections per user**, with guardrails on request-rejection rate, “I don’t know this person” feedback, blocks and spam reports.

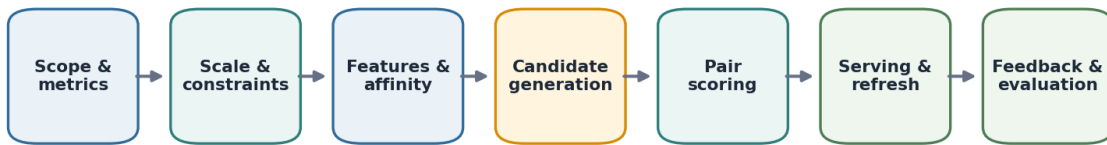
3. Model alternatives

Approach	Input	Strength	Cost
pointwise classifier / learning-to-rank (GBDT or MLP)	user, candidate and pair features, including hand-made graph features	easy to train, debug and serve	graph structure only through features you engineer
static graph model (node2vec / GraphSAGE embeddings) + recency features	graph snapshot + time-since features	learns structure automatically	a snapshot loses event order
temporal GNN (e.g. TGN, Rossi et al., 2020)	the stream of timestamped graph events	models <i>how relationships evolve</i>	per-node state, complex training and serving

Promotion rule: keep the simpler model unless event order and memory produce a measurable lift (e.g. $\geq 2\%$ relative mAP offline and a significant gain in accepted connections online) that pays for the extra state and complexity.

People You May Know design map

Move from product objective and graph scale to candidate generation, temporal scoring, serving, and feedback.



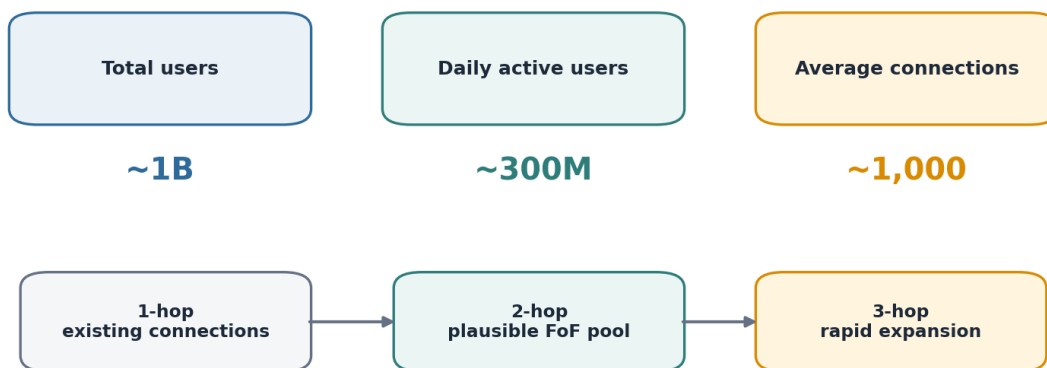
Minimum-scope principle

Begin with graph-based candidate generation and a pointwise or learning-to-rank baseline. Add temporal node memory only when recency features cannot capture the required lift.

Figure 29: People You May Know design map

Scale changes the architecture

Illustrative capacity-planning assumptions, not figures from any specific platform.



Design consequence: exclude current connections, cap graph expansion, deduplicate candidates, and rank before expensive pair modeling.

Figure 30: Scale constraints

Metrics separate attraction from mutual value

A request sent is not the same outcome as an accepted or durable connection.

Requests sent / received	Recommendation activity	Can reward low-quality outreach
Accepted requests	Mutual relevance	May still miss long-term value
Completed connections	Direct network growth	Sparse and delayed
Precision@K / mAP	Visible ranked-list quality	Offline proxy only
PR-AUC	Rare positive-edge discrimination	Does not model list position

Starting recommendation: gate the model with top-K and rare-positive metrics offline, then optimize accepted or completed connections online with safety and spam guardrails.

Figure 31: Metrics

Baseline versus graph-aware models

The temporal GNN is the richer proposal, not the required starting point.

Pointwise classifier / LTR - User, candidate, and pair features - Easy to debug and serve	Static graph model - Graph embeddings + recency features - Captures structure without node memory	Temporal GNN - Graph events + node memory + time encoding - Models evolving relationships
---	---	---

Promotion rule

Keep the simpler model unless event order and memory produce measurable lift that justifies state, training, and serving complexity.

Figure 32: Model alternatives

Build time-aware social affinity from several signal families

Mutual-connection count alone may miss recency, engagement, and repeated non-response.

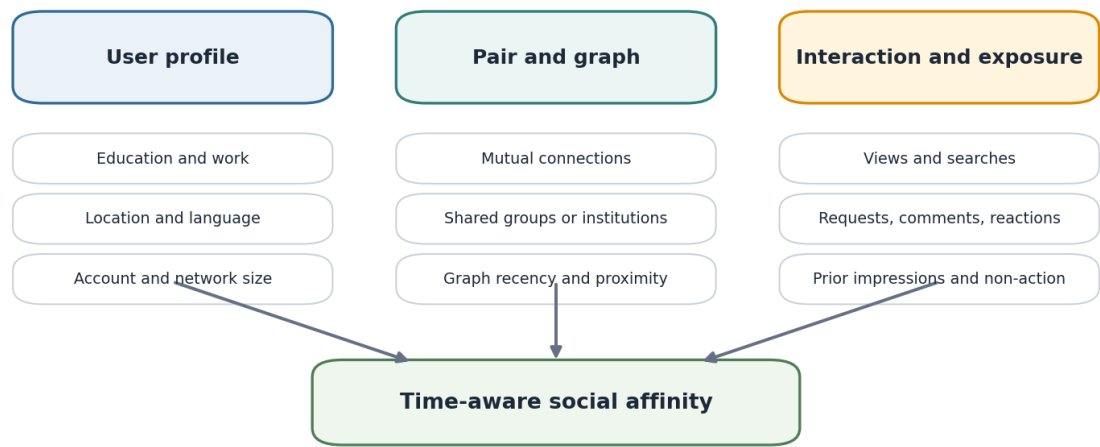


Figure 33: Features and affinity

4. Features: time-aware social affinity

Mutual-friend count alone is a strong but blunt signal. It misses recency and engagement, and it ignores people who have been shown repeatedly and ignored.

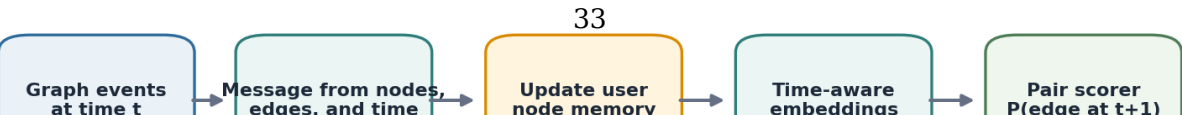
Family	Examples
user profile	school, employer, location, language, account age, network size
pair and graph	mutual connections (count and Adamic-Adar $\sum_{w \in N(u) \cap N(v)} 1/\log N(w) $, which down-weights mutuals who know everyone), shared groups or institutions, graph distance, recency of the mutual connections
interaction and exposure	profile views and searches between u and v , comments or reactions on each other’s posts, times v was already shown to u without action

Leakage trap. Every graph feature must be computed on the graph **before** the prediction time. Computing mutual-friend counts on today’s graph for a training pair that connected last week lets the model see the edge it’s supposed to predict, and its consequences.

5. The temporal GNN

Temporal GNN edge-prediction flow

Events update user memory, which produces time-aware embeddings for future-edge scoring.



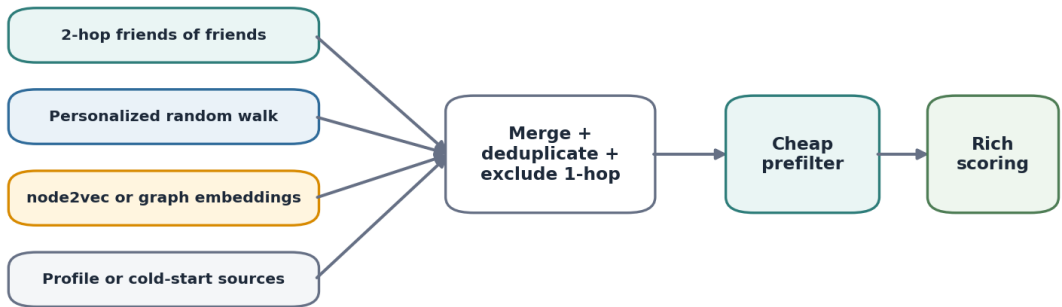
over recent neighbors then produces a **time-aware embedding**, and a pair scorer predicts $P(\text{edge}(u, v) \text{ at } t + 1)$.

The memory can hold recent graph changes, direct views and searches, engagement across the shared network, and time since meaningful events. The cost: every node carries state that must be updated in stream order, stored, and kept consistent between training and serving.

6. Candidate generation

Candidate generation should combine graph sources

Use inexpensive graph methods to produce a bounded pool before expensive scoring.



Friends-of-friends is usually the highest-yield source; the others add coverage and cold-start candidates.

Figure 35: Candidate generation

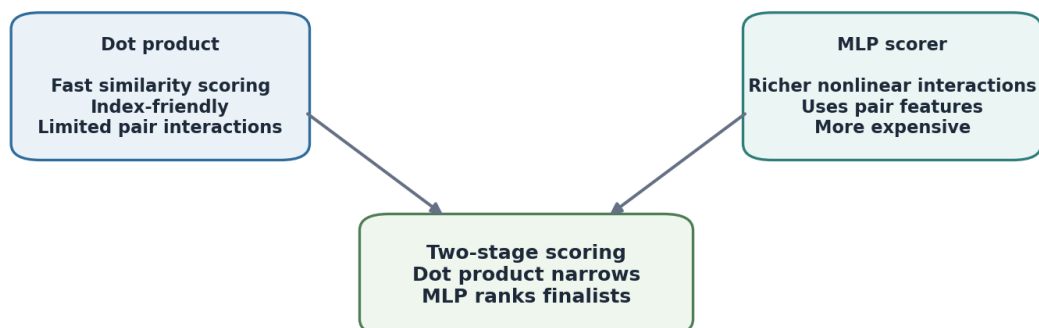
Source	Finds	Note
2-hop friends-of-friends	people with mutual connections	usually the highest-yield source; cap per user by mutual count
personalized random walk (PageRank with restart)	well-connected people 2-3 hops away	samples 3-hop without enumerating it
node2vec / graph-embedding ANN	structurally similar people, even without mutuals	catches communities
profile / cold-start sources	same school, company or city; contact-book matches (with consent)	for new users with few edges

Merge, deduplicate, **exclude existing connections, pending requests, blocked users and opted-out users**, then narrow with a cheap score.

7. Pair scoring

Dot product and MLP solve different scoring constraints

Use one or both depending on retrieval scale and pair-feature requirements.



Starting recommendation: use fast graph or embedding scores for candidate reduction, then use an MLP only where richer pair features provide measurable ranking lift.

Figure 36: Pair scoring

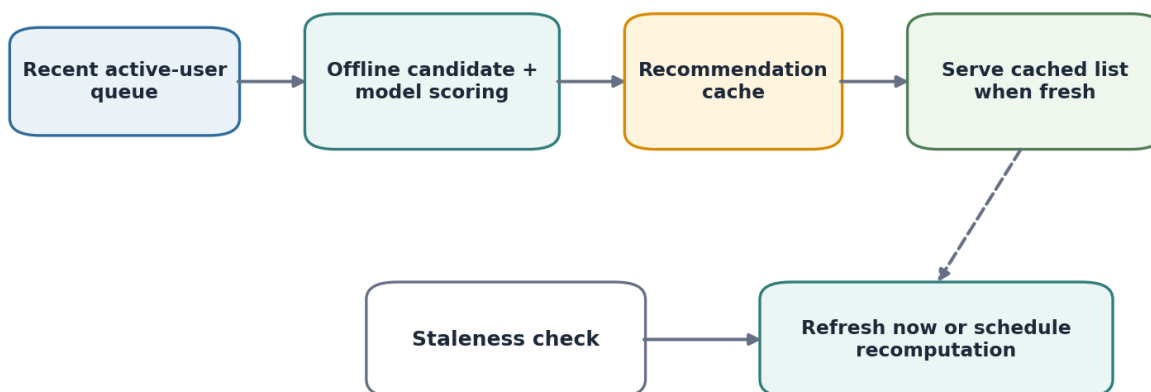
Scorer	Pros	Cons
dot product of embeddings	index-friendly, very cheap	no pair features, limited interactions
MLP on [user, candidate, pair features]	uses mutual counts, interaction history, exposure	more expensive per pair

Two stages: the dot product (or mutual count) narrows ~2,000 candidates to ~200, and the MLP ranks those finalists. Use the MLP only if pair features add measurable ranking lift.

8. Serving: precompute, cache, refresh lazily

Precompute, cache, and refresh lazily

Avoid recomputing recommendations after every small graph event.



A lightweight ranker may prefilter before expensive scoring, rerank cached candidates with fresh features, or perform both roles.

Missing profiles and repeated exposure need explicit treatment

Missingness is information; non-action becomes stronger only after repeated opportunities.

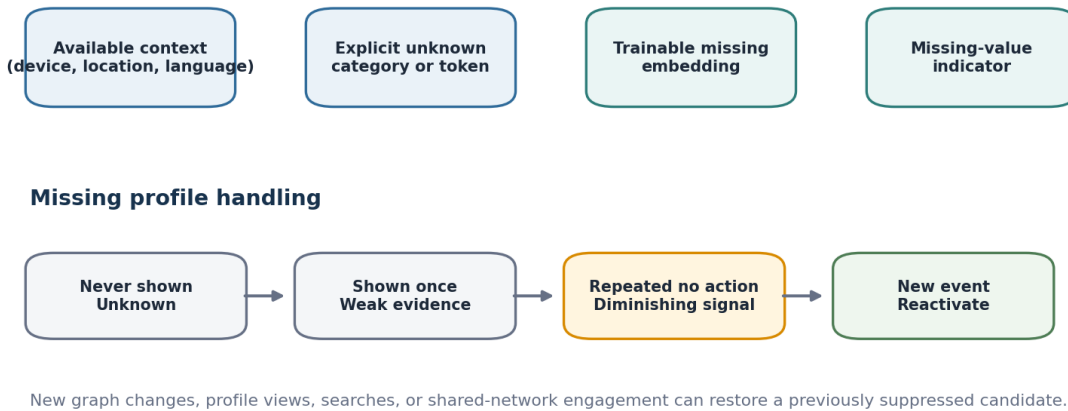


Figure 38: Missing data and exposure

Non-action is a graded signal:

never shown → unknown
 shown once → weak negative evidence
 shown repeatedly → diminishing score (e.g. multiply by $\gamma^{(\text{times shown})}$, $\gamma \approx 0.7$), then temporary
 new event → reactivate (new mutual friend, profile view, search, shared-group activity)

Delayed feedback: a request can be accepted days later. Evaluate and label with a window (e.g. accepted within 14 days), and don't train on pairs whose window hasn't closed.

10. Negatives and online evaluation

Negative type	Definition	Bias
random non-edges	any unconnected pair	too easy: most random pairs are obviously unrelated
graph-hard	friends-of-friends who didn't connect	realistic, but some are future positives
exposure	shown in PYMK, no action	the most relevant; depends on the old model's choices
time-aware	no edge within the label window	the correct labeling window, but delayed

Mix them: exposure + graph-hard for relevance, plus some random for calibration.

Online: *interleaving* compares two rankers quickly by merging their lists and counting accepts from each. *A/B tests* measure product impact.

Note. > **Beyond the book: network interference in A/B tests.** A PYMK change in treatment also affects control users, who receive the treatment users' requests. Randomize by graph clusters (ego-network or community randomization) or measure both sent and received effects. Otherwise the estimated lift is biased.

Training negatives and online comparison

Which non-connected pairs count as negatives shapes what the model learns.

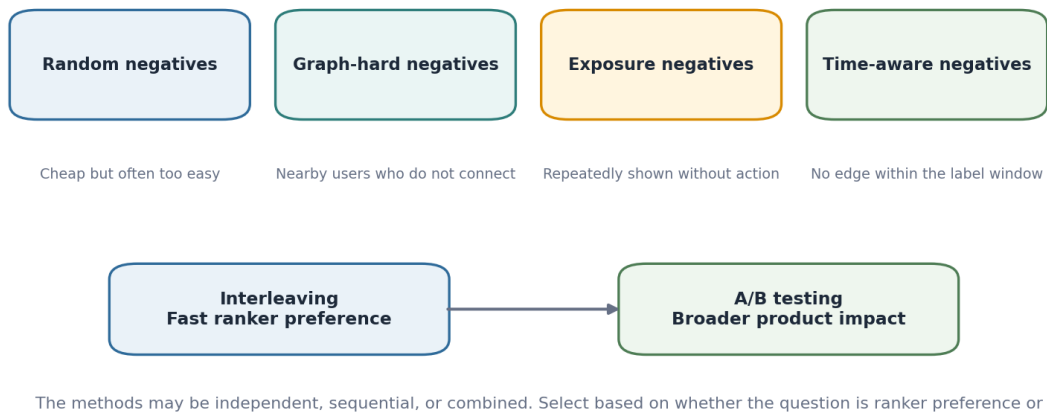


Figure 39: Negatives and evaluation

Note. > **Beyond the book: privacy.** Candidate sources like contact uploads or co-location can reveal sensitive relationships (a therapist, an ex). Respect privacy settings, avoid explaining recommendations with sensitive signals, and give users an easy “don’t suggest” control.

11. Reference architecture

Offline: graph and interaction event logs → FoF / random-walk / embedding sources → merge + dedupe + exclusions → pointwise (later static or temporal) scoring → cache writer with staleness metadata. **Online:** request → read cached candidates → fresh-feature light re-rank → exposure and policy filter → top-K.

12. Decisions and when I’d change them

Area	Starting choice	Switch when (measured)
product metric	accepted connections per user	long-term interaction data shows accepted-but-dormant links dominate → optimize connections that interact
candidates	capped 2-hop + random walk + cold-start sources	recall@K of the pool drops below target for low-degree users → add embedding ANN
model	pointwise GBDT/MLP with graph features	a temporal GNN beats it by $\geq \sim 2\%$ relative mAP offline and wins online
scoring	cheap narrowing → MLP finalists	the MLP adds no lift over the cheap score → drop it

Area	Starting choice	Switch when (measured)
serving	cached, lazy refresh, light re-rank	measured staleness hurts accepts (e.g. many suggestions of already-connected people) → refresh more often
ignored suggestions	decay by exposure count + reactivation	analysis shows re-shown candidates convert as often as fresh ones → weaker decay
evaluation	Precision@K / mAP / PR-AUC → interleaving → cluster-randomized A/B	-

13. Questions and answers

Q: Why not just rank by mutual-friend count?

It's a strong baseline, and should be the control arm. But it ignores recency (mutuals from 10 years ago), hubs (a mutual who knows everyone), engagement signals and prior ignores. Adamic-Adar and a learned model typically beat it.

Q: Should 3-hop candidates ever be used?

For users with few connections, yes, via random walks that sample 3-hop neighbors in proportion to connectivity. Enumerating 3-hop is never feasible.

Q: How much node memory can a temporal GNN afford?

At 10^9 nodes, 100 float32 values per node is already 400 GB of state. Memory has to be small (e.g. 32-64 dims), compressed, or kept only for active users.

Q: How should missing profile values be handled?

With an explicit unknown token and a missing indicator per field, so the model learns what missingness means (often "new user" or "privacy-conscious user"), rather than imputing a default value.

Q: Interleaving or A/B test?

Interleaving to pick between rankers quickly (it needs far less traffic). A cluster-randomized A/B test to measure the effect on network growth before launch.

Reference architecture

One production-balanced design under the current assumptions.

ONLINE SERVING



OFFLINE GRAPH LEARNING AND REFRESH



Starting recommendation: graph-based bounded candidate generation, a simple supervised scorer first, cached recommendations with lazy refresh, and temporal GNN promotion only after measurable incremental value.

Figure 40: Reference architecture

Chapter 4: Harmful Content Detection

Based on my study of *Machine Learning System Design Interview* (Aminian & Xu, 2023). The explanations, trade-off analysis and diagrams are my own. Notes marked **Beyond the book** are material I added.

The design in one paragraph. A post (text, images, video, links, context) goes through a cascade. First, cheap matching against known violating media. Then a **multimodal, multi-task model**: modality encoders → early fusion → shared layers → one head per harm category (violence, nudity, hate, self-harm, ...). The calibrated category scores feed a **policy layer** that combines degree and severity into an action: keep, demote, send to human review, or remove. It trains on large volumes of noisy user reports, is evaluated and calibrated on a smaller trusted human-labeled set, and goes to production shadow-first.

1. Problem definition

- **Input:** a newly created or edited post, with text, images, video, links, post context, and selected author and engagement features.
- **Model output:** a calibrated probability per harm category.
- **Service action:** keep / demote (limit distribution) / human review / remove (and possibly record a violation). Also structured **reason codes** for a separate service that writes the user-facing explanation.

post → [p_violence, p_nudity, p_hate, p_self-harm, ...] → severity-aware policy → action + reason

Related but separate systems:

- **Bad-actor detection** aggregates an account's posts, reports, violations and behavior over time into an actor-level risk score. It uses different features, labels and actions (e.g. account restrictions).
- **Misinformation** follows the same design process, but its labels depend on fact-checking and changing context. Out of scope here.

Chapter design map



Figure 41: Design map

From multimodal post to moderation action

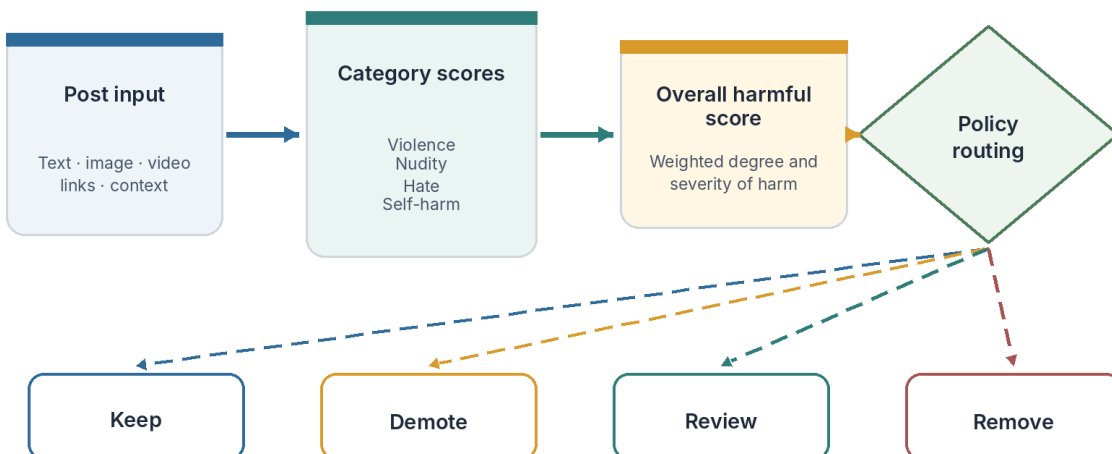


Figure 42: From post to action

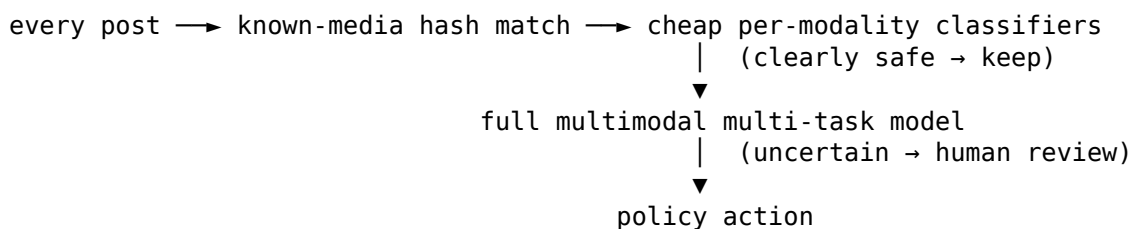
2. Requirements

Functional: many languages; text, image and video, alone or combined; category scores and one action; reason codes.

Latency depends on severity:

Tier	Categories (example)	Timing
real-time	graphic violence, credible threats, child-safety violations, self-harm with intent	before or within seconds of publication
near-real-time	nudity, hate speech	minutes; can be demoted while pending
asynchronous	lower-severity policy violations	batch re-scoring, e.g. as comments and reports accumulate

This leads to a **cascade**:



Note. > **Beyond the book: hash matching first.** Much harmful media is re-uploaded. Perceptual hashes (e.g. PhotoDNA, or Meta’s open-source PDQ for images and TMK+PDQF for video) match near-duplicates of already-confirmed violating content at almost no cost and very high precision. They’re the right first stage. The ML model is for *new* content.

3. Labels and training data

Two label sources, with different roles:

Source	Scale	Quality	Use
user reports	millions	noisy: mass reporting, disagreements, wrong category	weakly supervised training
trusted human labels (policy-trained reviewers)	thousands to tens of thousands	high	evaluation, calibration, threshold setting; limited fine-tuning

How I’d turn reports into labels:

- a single report is weak evidence. Weight it by the reporter’s historical accuracy;
- several independent reports raise confidence, but deduplicate coordinated reporting;
- if a moderator reviewed the report, use the moderator’s decision as a strong label, including “not violating” as a strong negative;
- unreported posts are *mostly* safe but not labeled negatives. Sample them into training with lower weight.

Label strategy: scale and trust play different roles

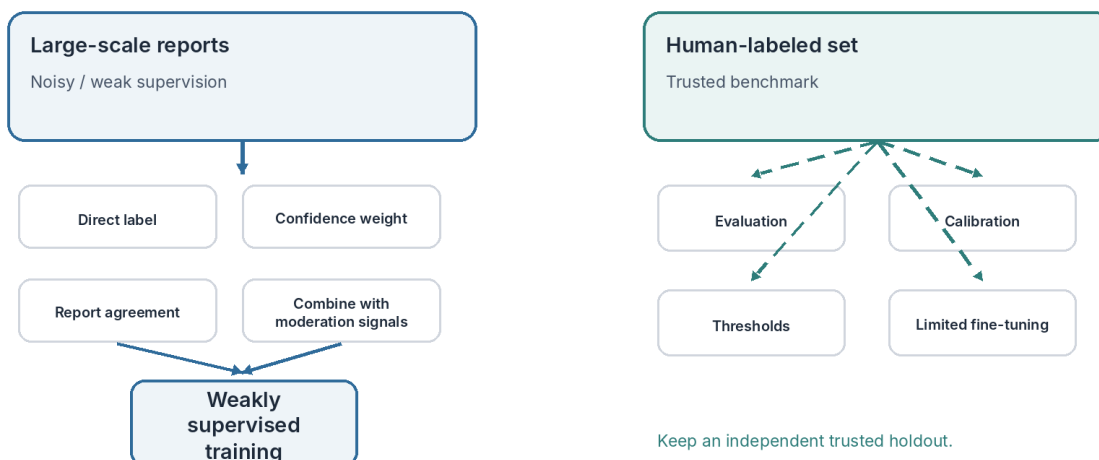


Figure 43: Label pipeline

Keep an independent trusted holdout. Never fine-tune on the evaluation set. I'd reserve about half of the human-labeled data, stratified by category and language, purely for evaluation and calibration.

4. Product metrics

Metric	Definition	Measures
prevalence (primary)	harmful impressions / total impressions, estimated by labeling a random sample of views	how much harm users actually see
content prevalence	harmful posts remaining visible / total posts	volume of harm on the platform
proactive rate	violating content removed before any user report / all violating content removed	how much the system finds on its own
valid-appeal (reversal) rate	enforcement decisions reversed on appeal / decisions appealed	wrongful enforcement
per-category and per-language breakdowns	each of the above, sliced	where the system is weak

Primary metric: impression-weighted prevalence, because a harmful post seen by a million people matters more than one seen by nobody. **Guardrails:** appeal reversal rate (over-enforcement), reviewer queue size and latency, creator retention.

5. Offline evaluation

- **PR-AUC and recall at a fixed precision** per category. Harmful content is rare (often < 1%), and precision-recall shows the positive-class trade-off directly.

- **ROC-AUC** is reported but not relied on. With millions of easy negatives, the false-positive *rate* stays tiny even when false positives outnumber true positives.
- **Calibration** per category, since thresholds and severity weights assume real probabilities.
- **Per-category thresholds.** A missed credible threat costs far more than a missed borderline insult, so the violence threshold for review is set lower. I'd set each threshold from the trusted set at a target precision for automatic removal (e.g. $\geq 95\%$) and a target recall for review.

6. Early vs late fusion

Multimodal fusion: modular baseline vs cross-modal learning

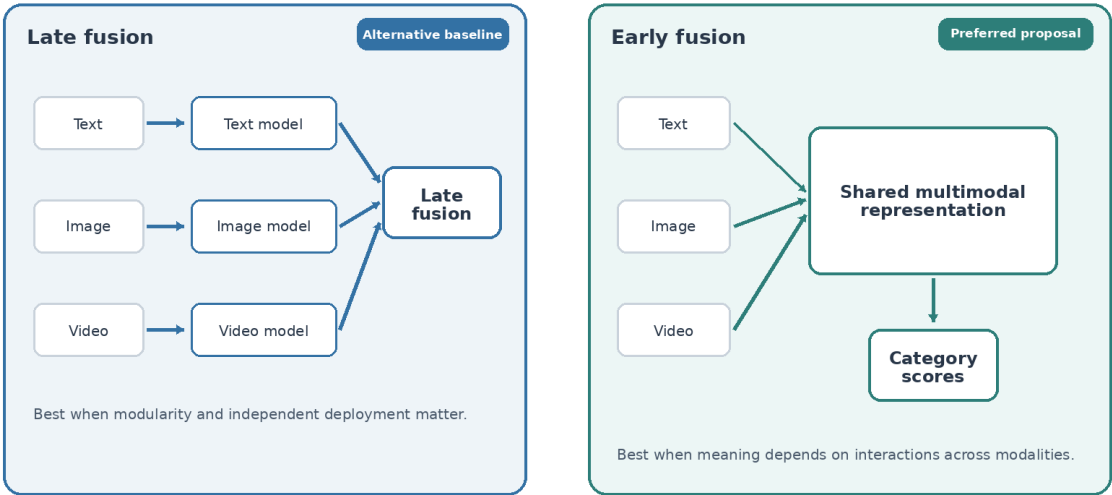


Figure 44: Fusion comparison

	Late fusion	Early fusion
structure	separate text / image / video classifiers, scores combined at the end	modality embeddings concatenated (or cross-attended) and fed to one shared model
strengths	modular, debuggable, each modality deployable on its own, missing modalities easy	learns cross-modal meaning
weaknesses	misses meaning that only exists in combination	harder to train; missing modalities need care; more compute

A harmless photo plus a harmless caption can be hateful together (the classic “hateful meme” problem), and late fusion can’t see that. So the **preferred design is early fusion, with late fusion as the baseline**. Adopt early fusion if it improves PR-AUC on a trusted set of multimodal posts (not just overall) by a meaningful margin.

7. Classifier structure

Classifier alternatives: from simple action to granular tasks

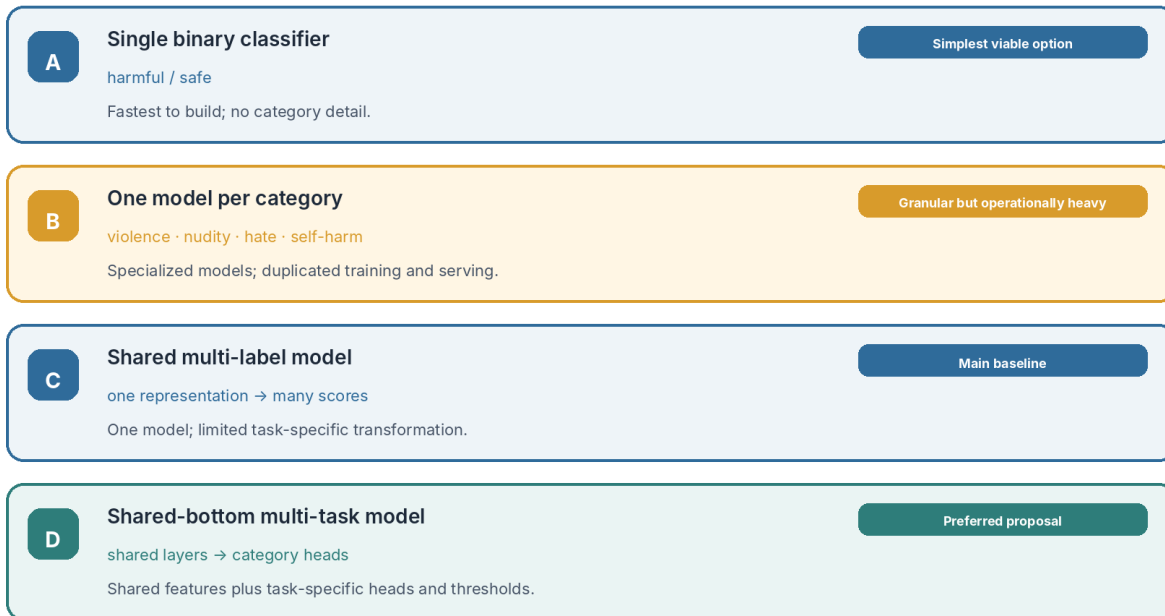


Figure 45: Model alternatives

Option	Description	Pro	Con
1. single binary	harmful vs safe	simplest	no category, so no category thresholds or reason codes N models to train, serve and keep consistent every category shares the same final layer
2. one model per category	separate violence, nudity, ... models	specialized	
3. shared multi-label	one network, one sigmoid output per category	one model, independent scores	
4. shared-bottom multi-task	shared layers + a small head per category	shared features <i>and</i> per-category transformations, thresholds and loss weights	more tuning; task gradients can conflict

Choice: option 4, with option 3 as the baseline. Give a category its own deeper head only if it underperforms when it shares, e.g. self-harm, whose language differs a lot from violence.

8. Modality encoders

Pretrained encoders, fine-tuned or frozen with trainable projection layers:

Modality	Options	Why
text	multilingual transformer: XLM-R, LaBSE, or a distilled multilingual model for latency	one model across ~100 languages; DistilBERT and Sentence-BERT are English-centric unless you use their multilingual variants
image	CLIP ViT image encoder; a ResNet/ViT pretrained with a self-supervised method like SimCLR	CLIP's image-text alignment helps text-in-image and meme understanding
video	sample frames → image encoder → temporal pooling or attention; or a video transformer (VideoMAE, X-CLIP), possibly pretrained with a self-supervised method like VideoMoCo	cost grows with length, so sample frames and add audio transcripts as text

(SimCLR and VideoMoCo are *training methods*, not architectures: they produce encoders.)

9. Missing modalities

Posts can be text-only, image-only, text + video, and so on. The model must tell “no image” apart from “a harmless image”.

Method	Pro	Con
zero vector	trivial	can look like a real low-information embedding
learned “missing” embedding per modality + a presence flag	the model knows what's absent	one extra parameter vector per modality
modality-specific handling (e.g. empty-string text encoding)	natural for some encoders	inconsistent across modalities

Choice: a learned missing embedding plus a presence flag for each modality, trained with **modality dropout** (randomly hiding a present modality during training) so the model stays robust when one is missing.

10. Additional features

Engagement (impressions, likes, shares, reports so far): useful for re-scoring, but zero at upload time. Only use values available at the moment of prediction.

Comments: they arrive after posting, so they don't help the first decision. I'd use them for **re-scoring**: mean-pool the embeddings of up to N recent comments (the simple default), or use attention pooling or max-risk pooling if a few alarming comments matter more than the average.

Author history (prior violations, report rate, account age, follower counts): strongly predictive, but it risks penalizing people for their past rather than this post, and it can encode bias. My choice: keep the harm score **content-only**, and use author history for **routing and prioritization** (e.g. faster review for repeat violators) and in the separate actor-level system. Only history before the current post may be used.

Context: posting time, device, country, surface (public vs group).

11. From category scores to an action

Several categories can be present at different levels. Options for combining them:

Method	Pro	Con
weighted sum $\sum_c s_c p_c$	interpretable	weights need policy input
max category	sensitive to one severe category	ignores several moderate ones
learned aggregator	captures interactions	opaque; needs overall labels
hybrid	policy control + flexibility	more to maintain

My choice (hybrid): per-category calibrated scores are checked against per-category thresholds **first**, so severe categories can trigger removal on their own. Otherwise, a severity-weighted risk $R = \max_c (s_c p_c)$, with severities s_c set by policy, plus a small bonus when several categories are moderately high, drives the demote / review decision.

12. Training the multi-task model

Training: balance tasks, modalities, and rare classes

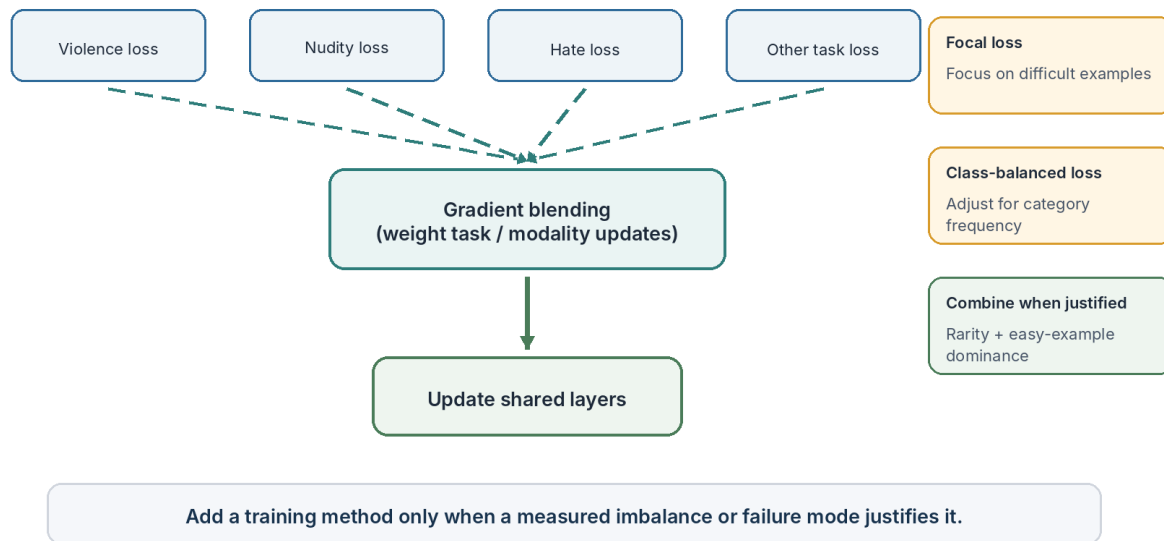


Figure 46: Training flow

Each head has a binary cross-entropy loss, and the shared layers receive the sum. Two different balancing problems are easy to confuse:

- **Balancing tasks:** some category losses are larger or noisier and dominate the shared gradients. **GradNorm** (Chen et al., 2018) learns loss weights that equalize per-task gradient norms. Uncertainty weighting (Kendall et al., 2018) is an alternative.
- **Balancing modalities:** in multimodal training, one modality (often text) is learned fast and overfits while others are under-trained. **Gradient Blending** (Wang, Tran & Feiszli, 2020) weights each modality's loss by its estimated overfitting-to-generalization ratio.

I’d first measure which imbalance exists (per-task gradient norms, per-modality validation curves), then apply the matching fix. Evidence that it helped: rare-category PR-AUC improves without a drop in the frequent categories.

13. Class imbalance

Harm is rare overall, and some categories are far rarer than others.

Focal loss (Lin et al., 2017) down-weights easy examples:

$$\text{FL}(p_t) = -\alpha_t(1 - p_t)^\gamma \log p_t, \quad \gamma \approx 2.$$

An easy negative with $p_t = 0.99$ has its loss scaled by $0.01^2 = 10^{-4}$. Risk: noisy reports look like “hard” examples and get *more* weight.

Class-balanced loss (Cui et al., 2019) weights class c by the inverse of its *effective number* of samples, $E_{n_c} = (1 - \beta^{n_c})/(1 - \beta)$. With $\beta = 0.9999$, a category with 100 examples gets about 100× the weight of one with 100,000, instead of the 1,000× that plain inverse frequency would give. That tames overweighting of noisy rare classes.

They address different things (difficulty vs rarity) and can be multiplied, but add each only when its failure mode is measured: easy negatives dominating the loss, or a rare category’s recall collapsing.

14. Long inputs and attention cost

Standard self-attention costs $O(n^2)$ in sequence length. Long posts, transcripts and many video frames get expensive. Options, in order:

1. **reduce n** : truncate, sample frames, pool patches (often enough);
2. **efficient attention**: linear or kernelized attention, low-rank (Linformer), or sparse/local windows. Each is cheaper and may lose some long-range interaction quality.

Switch when p99 latency or GPU memory is exceeded at the real length distribution, and accept the change only if category PR-AUC drops by less than a set tolerance.

15. Serving and moderation policy

A starting policy (thresholds on calibrated scores, set per category from the trusted set):

Condition	Action
any category $\geq$ its removal threshold (precision $\geq 95\%$)	remove; record a violation; send reason codes
severity-weighted risk in the review band, or model uncertainty high	human review queue, prioritized by severity $\times$ reach
risk in the demote band	limit distribution while pending
otherwise	keep

Severe categories get lower review thresholds, and some (credible threats) always go to review. Queue capacity is part of the design: if the review band sends more items than reviewers can handle, the threshold has to move.

Serving and rollout: risk-sensitive paths

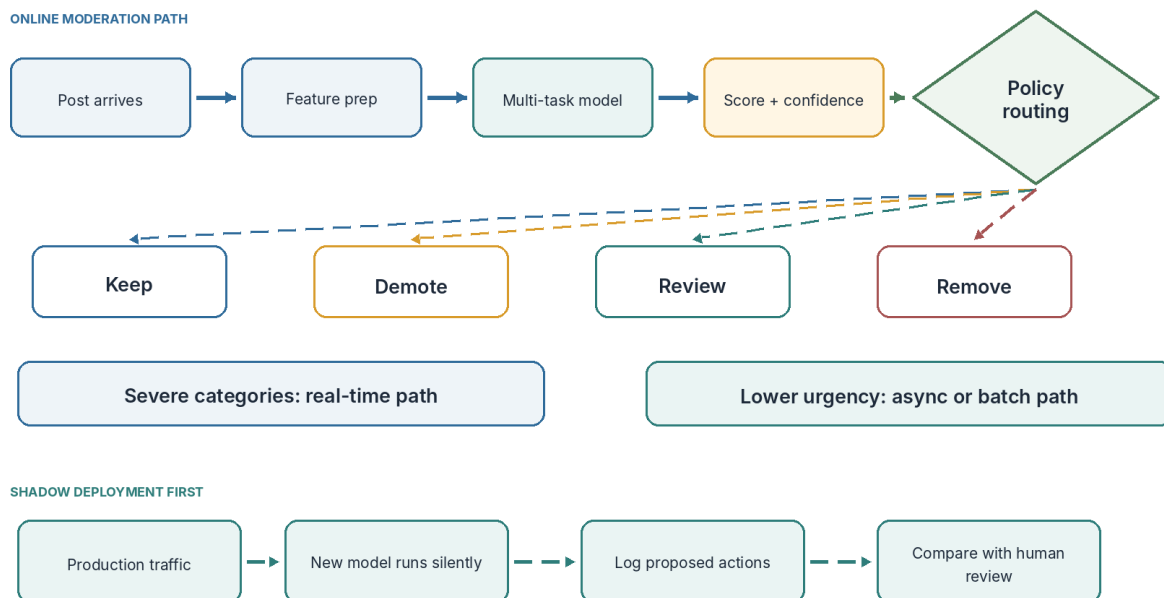


Figure 47: Serving and shadow deployment

16. Deployment: shadow first

A/B testing a moderation model means knowingly exposing some users to more harm, or wrongly removing their posts. So the first production step is **shadow deployment**:

production traffic → new model scores silently → proposed actions logged
 → compared with the current system and with human review of disagreements
 → limited enforcement (one category, one region) → full rollout

Shadow can validate latency, reliability, score distributions, agreement with production, the disagreement cases and per-category precision on reviewed samples. It **can't** measure how users respond to real enforcement. That comes with limited enforcement, and A/B tests are reserved for lower-risk changes (e.g. demotion thresholds).

Exit criteria I'd set: p99 latency within budget for two weeks; reviewed precision at the removal threshold $\geq$ production in every category; no category or language with recall below production; disagreement cases reviewed and explained.

17. End-to-end design

post → hash match → (match) → remove
 → text / image / video encoders (+ missing embeddings) → early fusion → shared layers
 → category heads → calibrated scores → severity-aware policy
 → keep / demote / review / remove (+ reason codes → explanation service)
 account history over time → actor-level risk (separate system)

18. My decisions

One reasonable design under the current assumptions

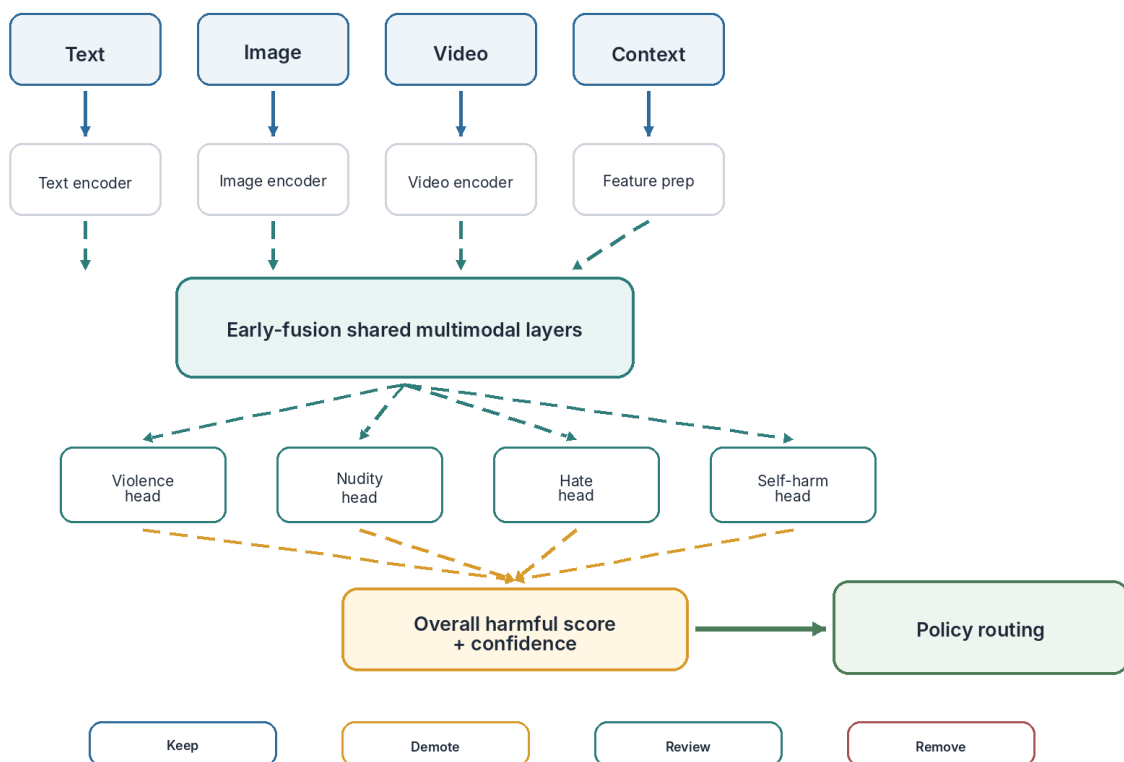


Figure 48: Reference architecture

Decision	Choice	Would change if...
main categories	violence, nudity, hate, self-harm, plus others by policy	policy adds or merges categories
real-time categories	violence, credible threats, child safety, self-harm with intent	latency cost is prohibitive → a cheaper real-time model for these only
user reports	reliability-weighted weak labels; moderator decisions as strong labels	a noise-robust loss beats weighting on the trusted set
human labels	half held out for evaluation/calibration, half for fine-tuning	-
primary product metric	impression-weighted prevalence	-
primary offline metric	per-category PR-AUC and recall at the removal-precision target	-
fusion	early fusion; late fusion as baseline	early fusion gains < ~1 pt PR-AUC on multimodal posts
architecture	shared-bottom multi-task; multi-label baseline	the baseline is within noise on every category
harm score	per-category thresholds first, then severity-weighted max	a learned aggregator beats it on overall labels and passes policy review
task / modality balancing	measure first; GradNorm for tasks, Gradient Blending for modalities	no measurable imbalance → plain summed loss
imbalance	class-balanced weights; add focal loss if easy negatives dominate	-
missing modality	learned missing embedding + presence flag + modality dropout	-
comments	re-scoring only, mean-pooled	a few alarming comments are missed → max-risk pooling
author history	routing and priority only, not in the harm score	policy decides author history is a legitimate input
rollout	shadow → limited enforcement → full	-

19. Main lessons

1. Multi-label inside, one action outside. Category scores enable thresholds, severity and reason codes.
2. Reports give scale; human labels give truth. Keep a trusted, independent evaluation set.
3. Early fusion is justified when meaning lives in the *combination* of modalities.
4. Balance tasks and modalities only after measuring which one is out of balance.
5. Focal loss addresses difficulty, class-balanced loss addresses rarity. They're different tools.
6. Represent missing modalities explicitly and train for their absence.
7. Moderation is a spectrum (keep, demote, review, remove), and reviewer capacity is part of the system.
8. Go shadow-first. Enforcement mistakes are costly in both directions.